

# Cross-Architecture Benchmarking of Lightweight and Post-Quantum Cryptography on Constrained IoT Platforms

Ethan C. Duval

Dissertation submitted to the Faculty of the  
Virginia Polytechnic Institute and State University  
in partial fulfillment of the requirements for the degree of

Master of Engineering  
in  
Computer Engineering

Timothy Talty, Chair

Jacek Kibilda

Nektaria Tryfona

April 27, 2026

Alexandria, Virginia

Keywords: Lightweight Cryptography, Post-Quantum Cryptography, IoT Security,  
Embedded Systems, Authenticated Encryption, Benchmarking, Ascon, ML-KEM, ARM

Cortex-M, RISC-V

Copyright 2026, Ethan C. Duval

# Cross-Architecture Benchmarking of Lightweight and Post-Quantum Cryptography on Constrained IoT Platforms

Ethan C. Duval

(ABSTRACT)

The proposed work presents ORBIT (Open-source Reference Benchmark for IoT Targets), an open-source, platform-independent benchmarking suite designed to assess five cryptographic algorithms: Ascon-AEAD128, Ascon-80pq, GIFT-COFB, AES-128-GCM, and ML-KEM-512 on five different hardware architectures, including the RP2040 Pico, nRF52840 DK, STM32 Nucleo F446RE, ESP32-C61, and Raspberry Pi 5. The assessment covers metrics such as cycles per byte, latency, energy usage, and timing variance. It has been shown that lightweight authenticated encryption algorithms are substantially more efficient than software AES-128-GCM on resource-constrained microcontrollers, achieving far better performance in terms of cycles per byte, latency, and energy cost. Ascon-AEAD128 and GIFT-COFB still perform very similarly to each other, whereas Ascon-80pq represents a slight increase in the symmetric overhead relative to Ascon-AEAD128. In turn, ML-KEM-512 exhibits the largest overhead in all analyzed parameters, making it feasible for periodic and infrequent tasks such as key exchange or refresh. The ESP32-C61 shows the best results across all metrics, with the STM32 standing out for timing performance and the nRF52840 DK for energy efficiency.

# Cross-Architecture Benchmarking of Lightweight and Post-Quantum Cryptography on Constrained IoT Platforms

Ethan C. Duval

(GENERAL AUDIENCE ABSTRACT)

In this project, ORBIT, which is an open-source benchmarking framework that can be used to measure the performance of cryptographic algorithms on multiple platforms, was proposed. In using ORBIT, five algorithms were tested on five different platforms, such as microcontrollers and Raspberry Pi 5, focusing on speed, power consumption, memory footprint, and timing consistency. According to the test results, lightweight algorithms outperform AES-128-GCM on microcontrollers in terms of power consumption and computation cost. It is seen that Ascon-AEAD128 and GIFT-COFB have similar performance results, whereas Ascon-80pq consumes extra resources by providing resistance to potential quantum attacks. ML-KEM-512, which aims at post-quantum key exchange, is possible yet costs significantly more, thus implying its usefulness only for sporadic key setups, not message encryption. Consequently, the results show that powerful and secure cryptographic algorithms can be implemented on constrained devices; however, it is understood that security mechanisms vary in cost due to their differences in type and purpose.

# Dedication

*To my family in the Cayman Islands, for everything.*

*And to Adele, for putting up with all of it.*

# Acknowledgments

I would first like to thank Dr. Talty for stepping in as my faculty advisor and guiding this project when I was in a difficult position. His willingness to take on this work, along with his feedback and support, made the project's completion possible.

I would also like to thank Dr. Jacek Kibilda for his early guidance on the direction of the project and for helping shape its initial scope. I am also grateful to Dr. Nektaria Tryfona and my full committee for their time, willingness to serve, and thoughtful engagement with this work.

I would like to thank Dr. Yi and her team, especially Salekin, for graciously allowing me to borrow the Analog Discovery 3 used in the energy measurements. I am also grateful to Marci Hyla for helping me gain access to the drone lab workspace, which made a substantial difference in carrying this project through.

Finally, I would like to thank Rebecca Stevenson for her continued support and help throughout these last few semesters. I am also deeply grateful to my family for their encouragement throughout this process, and to Adele for her patience, support, and for being there through all of the long days and late nights that went into completing this work.

# Contents

|                                                  |            |
|--------------------------------------------------|------------|
| <b>List of Figures</b>                           | <b>xi</b>  |
| <b>List of Tables</b>                            | <b>xii</b> |
| <b>1 Introduction</b>                            | <b>1</b>   |
| 1.1 Motivation . . . . .                         | 1          |
| 1.2 Problem Statement . . . . .                  | 5          |
| 1.3 Research Questions . . . . .                 | 6          |
| 1.4 Objectives & Contributions . . . . .         | 7          |
| <b>2 Review of Literature</b>                    | <b>9</b>   |
| 2.1 Constrained IoT Landscape . . . . .          | 9          |
| 2.2 Lightweight Cryptography . . . . .           | 11         |
| 2.3 Classical Cryptography in IoT . . . . .      | 13         |
| 2.4 Post-Quantum Cryptography Overview . . . . . | 14         |
| 2.5 Summary of Prior Work & Gaps . . . . .       | 17         |
| <b>3 Methodology</b>                             | <b>20</b>  |
| 3.1 Experimental Overview . . . . .              | 20         |

|       |                                                    |    |
|-------|----------------------------------------------------|----|
| 3.2   | Algorithms Under Evaluation . . . . .              | 20 |
| 3.3   | Hardware Platforms . . . . .                       | 23 |
| 3.4   | ORBIT Benchmarking Framework . . . . .             | 26 |
| 3.4.1 | Framework Architecture . . . . .                   | 26 |
| 3.4.2 | Platform Abstraction Layer . . . . .               | 27 |
| 3.4.3 | Host-Side Orchestration . . . . .                  | 27 |
| 3.4.4 | Benchmark Output Schema . . . . .                  | 28 |
| 3.5   | Benchmark Protocol . . . . .                       | 28 |
| 3.5.1 | AEAD Benchmark Procedure . . . . .                 | 28 |
| 3.5.2 | ML-KEM Benchmark Procedure . . . . .               | 29 |
| 3.5.3 | Correctness Validation . . . . .                   | 29 |
| 3.5.4 | Independent Runs . . . . .                         | 30 |
| 3.6   | Performance Measurement Method . . . . .           | 30 |
| 3.6.1 | Cycle Counting . . . . .                           | 30 |
| 3.6.2 | Cycles per Byte and Cycles per Operation . . . . . | 31 |
| 3.6.3 | Latency and Throughput . . . . .                   | 31 |
| 3.7   | Memory Measurement Method . . . . .                | 32 |
| 3.8   | Energy Measurement Method . . . . .                | 33 |
| 3.8.1 | Measurement Hardware . . . . .                     | 33 |
| 3.8.2 | GPIO Trigger Windowing . . . . .                   | 33 |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| 3.8.3    | WaveForms Trace Processing . . . . .              | 34        |
| 3.8.4    | Energy Computation . . . . .                      | 35        |
| 3.9      | Timing Consistency Analysis . . . . .             | 36        |
| 3.10     | Statistical Analysis . . . . .                    | 36        |
| 3.11     | Limitations . . . . .                             | 37        |
| <b>4</b> | <b>Experimental Procedure and Data Collection</b> | <b>39</b> |
| 4.1      | Collection Workflow . . . . .                     | 39        |
| 4.2      | Host Environment and Toolchains . . . . .         | 41        |
| 4.3      | Target Execution Procedures . . . . .             | 42        |
| 4.3.1    | Raspberry Pi Pico (RP2040) . . . . .              | 42        |
| 4.3.2    | Nordic nRF52840 . . . . .                         | 43        |
| 4.3.3    | STM32 Nucleo F446RE . . . . .                     | 43        |
| 4.3.4    | ESP32-C61 . . . . .                               | 44        |
| 4.3.5    | Raspberry Pi 5 . . . . .                          | 44        |
| 4.4      | Energy Trace Collection . . . . .                 | 45        |
| 4.4.1    | Measurement Setup . . . . .                       | 45        |
| 4.4.2    | Trace capture Procedure . . . . .                 | 46        |
| 4.4.3    | Trace Organization . . . . .                      | 47        |
| 4.5      | Post-processing and Dataset Assembly . . . . .    | 47        |



|          |                                              |           |
|----------|----------------------------------------------|-----------|
| 4.6      | Data Quality Checks . . . . .                | 48        |
| <b>5</b> | <b>Results</b>                               | <b>50</b> |
| 5.1      | Cross-Platform Performance Results . . . . . | 50        |
| 5.1.1    | AEAD Performance . . . . .                   | 50        |
| 5.1.2    | ML-KEM-512 Performance . . . . .             | 54        |
| 5.2      | Energy Results . . . . .                     | 55        |
| 5.2.1    | AEAD Energy Efficiency . . . . .             | 55        |
| 5.2.2    | ML-KEM Energy Cost . . . . .                 | 57        |
| 5.3      | Memory Usage Results . . . . .               | 58        |
| 5.4      | Timing Consistency Results . . . . .         | 60        |
| 5.5      | Post-Quantum Overhead . . . . .              | 61        |
| 5.5.1    | Ascon-80pq Overhead . . . . .                | 61        |
| 5.5.2    | ML-KEM-512 Overhead . . . . .                | 63        |
| <b>6</b> | <b>Discussion</b>                            | <b>64</b> |
| 6.1      | Tradeoff Analysis . . . . .                  | 64        |
| 6.2      | Practical Deployment Implications . . . . .  | 67        |
| 6.3      | Overall Limitations . . . . .                | 70        |
| <b>7</b> | <b>Conclusions &amp; Future Work</b>         | <b>72</b> |
| 7.1      | Conclusions . . . . .                        | 72        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 7.2 Future Work . . . . .                                         | 73        |
| <b>Bibliography</b>                                               | <b>75</b> |
| <b>Appendices</b>                                                 | <b>81</b> |
| <b>Appendix A Target Toolchain and Build Environment</b>          | <b>82</b> |
| <b>Appendix B Classical / Standardized Normalization</b>          | <b>84</b> |
| B.1 AEAD at 1024 B . . . . .                                      | 84        |
| B.2 ML-KEM-512 . . . . .                                          | 85        |
| <b>Appendix C Quantum Search Adjusted Symmetric Normalization</b> | <b>87</b> |

# List of Figures

|      |                                                                                                             |    |
|------|-------------------------------------------------------------------------------------------------------------|----|
| 5.1  | AEAD encryption cycles per byte at 1024-byte message size across all evaluated platforms. . . . .           | 51 |
| 5.2  | AEAD encryption latency at 1024-byte message size across all evaluated platforms. . . . .                   | 51 |
| 5.3  | AEAD encryption cycles per byte at 1024-byte message size for MCU-class platforms only. . . . .             | 52 |
| 5.4  | Ascon-AEAD128 encryption cycles per byte across message sizes for MCU-class platforms. . . . .              | 53 |
| 5.5  | ML-KEM-512 per-operation latency across all evaluated platforms. . . . .                                    | 54 |
| 5.6  | AEAD encryption energy per byte at 1024-byte message size for energy-measured embedded platforms. . . . .   | 55 |
| 5.7  | AES-128-GCM encryption energy per byte across message sizes for energy-measured embedded platforms. . . . . | 57 |
| 5.8  | ML-KEM-512 energy per operation for energy-measured embedded platforms.                                     | 58 |
| 5.9  | Ascon-80pq encryption energy per byte across message sizes for energy-measured embedded platforms. . . . .  | 62 |
| 5.10 | Ascon-80pq encryption cycles per byte across message sizes for MCU-class platforms. . . . .                 | 62 |
| 5.11 | ML-KEM-512 energy per operation for energy-measured embedded platforms.                                     | 63 |

# List of Tables

|     |                                                                                                                                         |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Hardware platforms evaluated in this study. . . . .                                                                                     | 23 |
| 4.1 | ORBIT data collection workflow. . . . .                                                                                                 | 40 |
| 4.2 | Common host environment and software used for data collection. . . . .                                                                  | 41 |
| 4.3 | Analog Discovery 3 signal connections for energy trace collection. . . . .                                                              | 45 |
| 4.4 | Data quality checks applied to each processed benchmark configuration. . . . .                                                          | 49 |
| 5.1 | Flash and RAM usage across MCU platforms and algorithms. . . . .                                                                        | 59 |
| 5.2 | Representative timing consistency results across platforms. . . . .                                                                     | 60 |
| 6.1 | Compact deployment-oriented suitability matrix derived from measured timing, energy, memory, and post-quantum overhead results. . . . . | 69 |
| A.1 | Target-specific toolchains and flashing dependencies. . . . .                                                                           | 83 |
| B.1 | Security-normalized AEAD efficiency at 1024 bytes under classical or standardized security assumptions. . . . .                         | 84 |
| B.2 | Security-normalized ML-KEM-512 efficiency under classical or standardized security assumptions. . . . .                                 | 85 |
| C.1 | Security-normalized symmetric AEAD efficiency at 1024 bytes under a quantum-search-adjusted normalization. . . . .                      | 87 |

# List of Abbreviations

AEAD Authenticated Encryption with Associated Data

AES-GCM Advanced Encryption Standard in Galois/Counter Mode

AVR Alf and Vegard’s RISC processor

BLE Bluetooth Low Energy

CAESAR Competition for Authenticated Encryption: Security, Applicability, and Robustness

CI Confidence Interval

CV Coefficient of Variation

DDoS Distributed Denial of Service

DSP Digital Signal Processing

DWT Data Watchpoint and Trace

ECC Elliptic Curve Cryptography

ECDH Elliptic Curve Diffie-Hellman

ENISA European Union Agency for Cybersecurity

FELICS Fair Evaluation of Lightweight Cryptographic Systems

FIPS Federal Information Processing Standard

FPU Floating-Point Unit

GIFT-COFB GIFT Cipher - Combined Feedback

IETF Internet Engineering Task Force

IoT Internet of Things

ISA Instruction Set Architecture

KEM Key Encapsulation Mechanism

LWC Lightweight Cryptography

MCU Microcontroller Unit

ML-KEM Module Lattice-Based Key Encapsulation Mechanism

NIST National Institute of Standards and Technology

NTT Number Theoretic Transform

ORBIT Open-source Reference Benchmark for IoT Targets

PQC Post-Quantum Cryptography

RAM Random-Access Memory

RFC Request for Comments

RSA Rivest-Shamir-Adleman

SDK Software Development Kit

SPN Substitution-Permutation Network

SRAM Static Random-Access Memory

TLS Transport Layer Security

# Chapter 1

## Introduction

### 1.1 Motivation

Over the course of the last decade, IoT has rapidly expanded, with some analysts estimating that the number of IoT devices is in the tens of billions [1], and will likely continue to rapidly grow. These devices are deployed in a wide range of environments, from industrial applications like pipeline pressure sensors, to wearables that track biometric data, to smart home devices like video doorbells, smart door locks, and even robot vacuum cleaners that have made their way into every day lives for millions of people. No matter what the nature of the deployment is they all struggle with the same core issue: extreme resource constraints.

Given the inherently resource bound nature of the deployment of IoT devices, they are not afforded the same memory and power availability of more traditional computational end points such as laptops, desktop workstations, or cloud servers. Some IoT devices run entirely on coin cell batteries, and need to be able to last for months if not years with little to no intervention, leaving extremely thin margins for extra compute power. From a cost perspective, certain microcontrollers need to be deployed to ensure that profit margins are met, meaning these devices often end up running on more affordable chips that are built on 8-bit or 32-bit architectures, with RAM numbers in the low kilobytes, compared to gigabytes we are used to seeing.

Security is a core technical challenge here, with impacts on privacy, public safety, and even national infrastructure. Devices like this are often "set and forget", once deployed they are a permanent, often unsecured, device on the network. In recent years, attacks such as the Mirai botnet attack in 2016 affected thousands of IoT devices, making use of them to launch wide scale DDoS attacks of scale never before seen [2]. More recently, in November of 2025, Cloudflare experienced and mitigated an attack that sustained 31.4 Tbps for 35 seconds, using an Aisuru-Kimwolf botnet composed of millions of Android TVs [3]. Due to events like this in the last few years, regulatory bodies such as NIST and ENISA have scrutinized IoT devices' security in both home and medical environments, leading to a more regulated security standard across the board [4, 5].

The classical answer to the security problem has long been using the industry established encryption protocol AES-128-GCM, though this poses some rather large challenges when brought into the realm of resource constrained embedded systems. AES-GCM has gone through rigorous testing and is one of, if not the most vetted encryption standard, and is at the core of the likes of TLS 1.3 and vastly recommended by NIST [6], but its primary use scenario involves having access to hardware acceleration, which is not always the case in these environments. This leads to the use of software based AES, which imposes computational overhead beyond reason for these devices, increases latency to levels not suitable for real time processing, and draws power in excess of what regular battery powered systems can handle [7]. In addition to these issues, there is also a purely security risk when it comes to AES, revolving around its reliance on lookup tables when run on purely software, allowing for potential cache-timing side channel attacks, which remain an issue even on systems without traditional caches due to branch timing variations [8].

In response to these shortcomings of traditional cryptographic algorithms, NIST set forward with a multi-year campaign to standardize lightweight cryptography, requesting algorithms



from the community designed specifically for resource constrained environments. Spanning three main rounds of testing and 57 initial algorithm submissions, looking into properties such as cryptanalytic security, implementation efficiency, and side-channel resistance, NIST announced the Ascon family of algorithms as the winner in 2023 [9], becoming the standard for AEAD and hashing for LWC. Ascon makes use of a sponge-based permutation design, which allows for a compact hardware footprint, low RAM requirements, and software performance characteristics that are favorable on the likes of 32-bit microcontrollers [10].

In addition to classical lightweight cryptography, an up and coming threat changing the landscape of security in IoT environments, is the development of cryptographically relevant quantum computers. If Shor’s algorithm were to be executed on a capable quantum computer, it would render all current public-key cryptography such as RSA, ECC, and ECDH entirely insecure [11]. While this technology does not yet exist, there is a credible risk of encrypted data being collected now, and decrypted once the ability comes to market, this is extremely worrying in deployments where long term confidentiality is required [12]. The reason that this is relevant to IoT is that most devices in industrial or medical contexts could have an expected lifetime of ten or even twenty years, potentially within the window that many analysts place the threat of quantum coming to fruition.

In 2016 NIST published a report that acknowledged the possibility that quantum technology could render RSA entirely insecure by 2030 [13], leading to an effort to standardize a suite of encryption standards, designed for a post-quantum landscape. This standardization process ran through four extremely competitive rounds spanning eight years, starting in 2016 with 82 submissions and 69 algorithms accepted to move into the first round. In 2022 NIST announced the final four candidates, CRYSTALS-Kyber, CRYSTALS-Dilithium, Sphincs+, and FALCON, and in August of 2024 NIST finalized the first three standards, with Kyber being designated the primary, and FALCON aimed to be standardized at a later date [14,

[15](#), [16](#)]. In March of 2025 HQC (Hamming Quasi-Cyclic) was selected as a backup KEM to ML-KEM (CRYSTALS-Kyber). NIST has since published a document stating that quantum-vulnerable algorithms should be deprecated from NIST standards by 2035, with high-risk systems being required to transition much earlier [\[12\]](#).

While the standardization of ML-KEM and the NIST PQC suite prove to be a significant milestone on the road to securing data in a post quantum world, standardization only represents half the battle. The deployment of these standards presents quite the challenge when it comes to constrained embedded systems. These algorithms were understandably evaluated with security and correctness criteria as the main focus, constrained embedded hardware was not a focus of this standardization effort. Where desktop and server deployments have the available compute capabilities, and access to power, smaller embedded systems cannot make the same assumption. ML-KEM is RAM intensive by nature, relying on lattice based operations, and when it comes to systems that are working with 16-256KB of RAM sometimes this presents a significant road block. On top of this, ML-KEM-512's public-keys and ciphertext are much larger when compared to classical cryptographic algorithms, which in the case of IoT devices, could lead to a struggle to stay within compute and transmission budgets. Comparing these PQC algorithms to the likes of AES-GCM, one of the issues we currently see is the lack of hardware acceleration built onto modern MCUs, meaning that these algorithms would need to run entirely through software. While in practice it may be trivial to implement these PQC algorithms onto the likes of a server, due to these caveats their feasibility on an embedded target still remains unknown, meaning empirical hardware testing must be completed. While PQC benchmarking projects exist such as the pqm4 project which covers Cortex-M4 [\[17\]](#), they do not include classical AEAD comparison or energy measurement. No published work evaluates ML-KEM alongside classical LWCs like Ascon or GIFT-COFB, across multiple architectures using a unified methodology. Bridging

this gap would give IoT designers the necessary data to be able to make informed decisions on developing IoT hardware for a safe, post-quantum world. This thesis addresses this, providing a baseline of performance, energy, and memory data that spans classical, and post-quantum cryptographic landscapes on popular constrained hardware.

## 1.2 Problem Statement

No unified evaluation framework exists with the purpose of benchmarking Classical cryptography, Lightweight cryptography, and post-quantum cryptographic algorithms across currently deployed and popular constrained embedded systems. These devices are often designed to exist in extremely constrained environments with little intervention, yet the algorithms classically developed for them are not developed with the same constraints in mind. NIST has recently standardized both LWC with Ascon, and PQC with ML-KEM, which has increased the need for cross platform evaluation of these and other similar algorithms on frequently deployed constrained environments.

FELICS as well as the NIST candidate evaluations themselves are good examples of prior benchmarking work, primarily look at individual algorithms on limited platforms, reporting cycle counts and code size. They mostly omit energy usage, and predate the actual standardization of Ascon and ML-KEM. Pqm4 also exists as an example of a post-quantum benchmarking paper which looks only at performance on Cortex-M4, and does not compare against classical AEAD or look into any physical energy measurements. No published work evaluates Ascon-128, Ascon-80pq, GIFT-COFB, AES-128-GCM, and ML-KEM-512 side by side on the same hardware under identical test conditions while quantifying security-efficiency tradeoffs. Without cross-platform empirical data, selecting the correct cryptographic primitive for constrained deployment remains largely guesswork.

This project aims to fill this gap directly, implementing Ascon-128, Ascon-80pq, GIFT-COFB, AES-128-GCM, and ML-KEM-512 on the same five target platforms, from ARM Cortex-M0+ to RISC-v based ESP32-c61, measuring performance, energy, and memory numbers that embedded designers actually need.

### 1.3 Research Questions

This report is guided by one main research question and three supporting questions. The primary question defines the overall scope of the research, while the supporting questions address specific measurement dimensions across the evaluated algorithms and architectures.

**PRQ:** To what extent are NIST-standardized lightweight and post-quantum cryptographic algorithms computationally and energetically viable on resource-constrained IoT microcontrollers, and how do their performance, energy, and memory characteristics compare to AES-128-GCM across multiple embedded architectures?

**SQ1:** How do Ascon-128, GIFT-COFB, and AES-128-GCM compare in throughput, latency, cycles per byte, and memory footprint across ARM Cortex-M0+, Cortex-M4, and RISC-V platforms?

**SQ2:** What is the energy cost per cryptographic operation for each evaluated algorithm across target platforms, and how does this cost scale with increasing message size?

**SQ3:** How do the post-quantum primitives Ascon-80pq and ML-KEM-512 compare to their classical counterparts in computational overhead, energy consumption, and memory footprint, and what do these results imply for post-quantum readiness on constrained embedded hardware?

## 1.4 Objectives & Contributions

The following objectives define the scope of this report and collectively address the primary research question and its supporting questions.

1. **Open-Source Benchmarking Framework.** The main objective of this project is to develop ORBIT (**O**pen-source **R**eference **B**enchmark for **I**oT **T**argets), an open-source portable C framework designed for bare-metal AEAD and KEM Benchmarking. It is designed with modularity in mind, allowing future extension for additional algorithms and platforms. Having a consistent methodology across all boards and algorithms is core to the design of this project.
2. **Multi-Platform Empirical Dataset.** This work develops a multi-platform, multi-algorithm dataset covering timing, energy, and memory. Each configuration is evaluated across five independent runs, with iteration counts adjusted by workload and platform rather than fixed uniformly, to accommodate heavier cryptographic workloads while preserving stable aggregate results. Results are reported as mean, standard deviation, and 95% confidence intervals. This dataset will be made publicly available upon completion of this research.
3. **Security-Efficiency Metrics.** This work derives security-normalized efficiency metrics, including security bits per cycle and security bits per microjoule, from the raw output data obtained through quantitative testing. These metrics allow for comparison across families that is not feasible using raw cycle counts alone.
4. **Deployment Suitability Matrix.** Using the data obtained throughout this research, this work develops a suitability matrix across different deployment profiles, covering wearables, smart home, industrial, gateway, and PQ-aware endpoints. This is aimed

at providing designers with practical decision-making information rather than raw numbers alone.

5. **Post-Quantum Feasibility Assessment.** This research assesses how feasible it is to deploy ML-KEM-512 and Ascon-80pq on each of the 5 target platforms, under the power, memory, and compute budgets seen in real IoT deployments. NIST has mandated the deprecation of quantum-vulnerable algorithms by 2035, and IoT devices commonly have lifespans of ten to twenty years in industrial and medical applications. Hardware that is deployed and secure today may still be active when that deadline arrives. By measuring the real computational and energy costs of these post-quantum primitives on constrained IoT hardware, this research offers designers a practical answer to whether or not that transition is feasible on the hardware they are actually shipping.

# Chapter 2

## Review of Literature

### 2.1 Constrained IoT Landscape

The IETF has published a formal taxonomy for resource-constrained nodes in RFC 7228 [18]. RFC 7228 defines constrained nodes with multiple limiting factors, including constraints on ROM and Flash for code complexity, RAM for state and buffer sizes, processing power, available power, and deployment accessibility for tasks such as setting keys or updating the device's software. Along with providing information on what "constrained" means in a deployment sense, RFC 7228 also separates devices into three main classes: Class 0 with well under 10 KiB of RAM and under 100 KiB of Flash, Class 1 with around 10 KiB of RAM and around 100 KiB of Flash, and Class 2 with around 50 KiB of RAM and around 250 KiB of Flash. The boards selected for this research map neatly to these classes: The RP2040 sits at the Class 1/2 boundary, the STM32 is solidly in Class 2, and the ESP32-C61 and nRF52840 are above Class 2.

Constrained devices are found almost everywhere, from consumer environments to critical national infrastructure, and their security requirements remain regardless of the deployment context. Devices like pipeline sensors, smart grid nodes, and industrial control systems have long operational lifespans and are commonly deployed and left as a permanent fixture on the network. The Mirai botnet is a prime example of what happens when these devices are unmaintained and unsecured [2]. Implantable devices and continuous health monitors

need to preserve power as strictly as possible, increasing the consequences from reduced battery life, to potential patient harm: having to replace a glucose monitor daily compared to bi-weekly could lead to serious concerns. Traditional Geostationary satellites frequently operate past their stated lifespan of 15 to 20 years, and are physically inaccessible the entire time they are deployed [19]. CubeSats and other small satellites have much shorter lifespans of 2 to 5 years [20], and often run on commercially available off the shelf microcontrollers directly comparable to the devices evaluated in this research, making them a relevant target for implementing lightweight cryptography. One thing that is common between all of these deployments is the fact that the algorithm that it is deployed with, will likely be one that it will use for the entirety of its operational life, meaning that its effects extend far beyond the device's initial deployment window.

Power budget eliminates the possibility of using software-heavy algorithms. Given battery powered devices very often operate on milliwatt and even microwatt power budgets and the fact that every cryptographic operation draws from this power budget, the overhead alone of running AES-128-GCM would meaningfully reduce the device lifetime [7]. This means that an algorithm that is too energetically expensive simply cannot be used no matter how secure it is in nature. Class 0 and Class 1 devices work with RAM budgets of Kilobytes, not Gigabytes or even Megabytes, and in cases of algorithms with large key schedules, heavy state requirements, or large public keys, they may not physically fit in available RAM. This is extremely relevant in the case of algorithms such as ML-KEM-512 which has public keys and ciphertext much larger than classical AEAD. The fact that many IoT devices operate under real-time or near real-time requirements, must also be taken into consideration. In the case of a time sensitive industrial control application, a device that takes 500ms to encrypt a reading before its transmission may be fundamentally useless. Without hardware acceleration, computationally heavy algorithms implemented purely as software would push



latency beyond acceptable thresholds. Performance under energy consumption, memory footprint, and latency form the core measurement framework for this research.

## 2.2 Lightweight Cryptography

General purpose cryptographic standards often assume computational resources that are not afforded to constrained devices, this not only presents a performance concern, but brings about questions on viability as a whole. This is where Lightweight Cryptography comes in. The design philosophy of LWC centers on minimizing state size, avoiding lookup tables, requiring no hardware acceleration to operate properly, and efficiency in both hardware and software implementations. NIST formally recognized this gap, leading to a multi-year LWC standardization effort.

NIST initially began investigating constrained cryptography in 2013, and published a formal call for LWC submissions in 2018. They received a suite of 57 submissions in March 2019, beginning the first round of testing in April 2019. They evaluated each submission based on three criteria: cryptanalytic security, implementation efficiency on constrained hardware, and side channel resistance. The analysis followed a similar community-driven multi-round public cryptanalysis process that AES went through, providing credibility via years of open scrutiny. In 2021 NIST announced 10 finalists, eventually selecting the Ascon family in February of 2023 [9], formally standardized as NIST SP 800-232 in August of 2025 [21, 22].

Ascon itself was developed initially in 2014 by a team from Graz University of Technology, Infineon Technologies, and Radboud University. It was also selected as the primary lightweight AEAD algorithm in the CAESAR competition in 2019, meaning it survived many years of cryptanalysis even before NIST had selected it. The algorithm is a sponge-based design using the Ascon-p permutation on a 320-bit state, using this single permutation to support both

encryption and authentication, keeping its implementation footprint extremely compact [10]. Ascon uses no lookup tables, eliminating the timing side channel vulnerabilities that affect AES. Its state fits within the register file of a 32-bit MCU, meaning no external memory access is needed during permutation rounds, which is critical for Class 1 and Class 2 devices. Ascon has several variants, though we will focus on two: Ascon-128 which is the primary AEAD variant with a 128-bit key, and Ascon-80pq which is a 160-bit key variant designed to be more quantum resistant. In regards to Grover's algorithm, which effectively halves symmetric key security, Ascon-80pq's 160 bit-key provides 80 bits of security compared to the 64 bits Ascon-128 would provide against a sufficiently equipped quantum adversary [23]. Ascon-80pq is not designed nor is it intended to replace ML-KEM for key encapsulation, but it does provide quantum resistant symmetric AEAD, without altering the underlying algorithm.

Another one of the NIST top 10 LWC finalists was GIFT-COFB. While not ultimately selected, GIFT-COFB was studied extensively and became well understood throughout the selection process. This algorithm has an underlying primitive of GIFT-128 which is a 128-bit block cipher that utilizes a substitution-permutation network design [24]. Implemented and designed as a successor to PRESENT, which is one of the earliest recognized lightweight block ciphers. Building off of PRESENT, GIFT was designed to reduce hardware gate count while improving the security properties of the algorithm. The second part of the algorithm name, COFB, stands for Combined Feedback, due to the fact that this wraps GIFT-128 into an AEAD scheme by way of using a feedback-based construction needing only one block cipher call per input block, making it both efficient and inverse-free during decryption. GIFT-COFB had one of the smallest hardware footprints among the 10 finalists. When it came to the software side, its bit permutation layer made it much more computationally complex when it came to software environments, making the software versus hardware tradeoff much

more pronounced when compared to Ascon [25].

While Ascon and GIFT-COFB represent the state of the art when it comes to Lightweight Cryptography, neither displaces AES-128-GCM against which all cryptographic performance is ultimately measured.

## 2.3 Classical Cryptography in IoT

AES-GCM-128 is a combination of two fundamental components, AES-128 as the foundational base block-cipher, with Galois/Counter Mode on top acting at the authenticated encryption mode of operation. AES has been the global standard when it comes to symmetric encryption since its standardization in 2001, and continues to be the center of secure communication to this day, acting as the core of protocols such as TLS 1.3, IPsec, and SSH. With GCM, AES becomes an AEAD scheme due to the fact that it completes authentication on top of encryption, in a single pass. The inclusion of GCM is one of, if not the, main reason that AES is so effective on both hardware and software [6]. AES acts as the natural basis for this research, as any cryptographic algorithm that comes after it, whether lightweight, post-quantum, will need to be compared to it in regards to security, and general applicability.

AES was initially designed with the assumption that it will have access to hardware acceleration. Most modern devices like servers, laptops, and even smartphones, have some form of AES-NI or similar hardware instructions built in, to take advantage of AES in its hardware accelerated state. When it comes to the microcontroller space this often breaks down due to the constrained environments sometimes not allowing for AES acceleration at all. Software AES relies heavily on lookup tables, for example the SubBytes operation requires a 256-byte S-Box operation that must be accessed repeatedly throughout rounds, and on deployments

with 10-50 KiB of RAM, that is not exactly trivial to accomplish. These repeated table lookups also present another major issue when it comes to computational overhead. Software AES on Cortex-M0+ can take sometimes hundreds of cycles per byte compared to the single digit cycle counts when using hardware [7]. These extra cycles are also directly reflected when it comes to energy overhead: and on a coin-cell battery powered system, this could meaningfully reduce the battery life.

Cache-timing side channel attacks are another issue that arises with AES due to its reliance on lookup tables. When AES accesses its S-Box lookup tables, the memory access pattern is based on the key and the plaintext that is being produced. In the case of a processor with a traditional cache, an attacker can observe timing variations over multiple encryptions, and using statistical analysis, recover key material. On systems missing a traditional cache, this is still an issue, due to branch timing variations and memory access latency, can provide similar vectors for timing based key material recovery [8]. This is precisely one of the major benefits of using Ascon on constrained systems, because of its sponge based permutation design, it relies on a fixed state removing the possibility for timing variation attacks. Despite these shortcomings, AES-GCM still remains the dominant cryptographic backbone of computational security. Though neither AES-GCM nor the NIST LWC algorithms were designed to combat the emerging threat of cryptographically relevant quantum computers.

## 2.4 Post-Quantum Cryptography Overview

Classical public key cryptography like RSA, ECC, and ECDH rely on the computational hardness of integer factorization and discrete logarithm problems, both of which are solvable in sub-exponential to exponential time on classical hardware though are reduced to polynomial time complexity by Shor's algorithm on a capable quantum computer [11]. Symmetric-

key cryptography is also affected by quantum, though less catastrophically. Grover’s algorithm allows for a quadratic speedup in the case of brute force key search, effectively halving symmetric keys security level [23]. This is precisely the reason that Ascon-80pq, with its 160-bit key, is a meaningful quantum hedge for symmetric AEAD. Though, ML-KEM is still required as the next step, in order to address the problem of key encapsulation.

While the concept of a ”sufficiently capable quantum computer” is still a future-facing issue, adversaries can still collect and store encrypted traffic today for decryption once this capability arrives [12]. This is predominantly an issue in long-term data storage that requires high levels of confidentiality: medical records, industrial control communication, military and classified government communications [26], and long-lived credentials. When it comes to industrial and medical contexts, IoT devices can have lifespans of ten to twenty years, potentially still transmitting sensitive data when cryptographically relevant quantum computers arrive. Geostationary satellites present a particularly interesting case; their hardware architecture is physically fixed at the point of manufacture, including cryptographic accelerators and onboard processors, meaning a satellite designed with hardware-accelerated AES in mind, will likely not be capable of running ML-KEM, as it is missing NTT acceleration. While some satellites do receive updates to their software over the course of the 15 to 20 years they are in service, the underlying computational capabilities often rely on hardware that is impossible to change after deployment [19]. Because of this satellites that are launched today will very likely still be in active service when the 2035 NIST deprecation deadline arrives [12].

In 2016 NIST published a report acknowledging that quantum computers could effectively render RSA insecure by 2030, this marked the start of a formal PQC standardization effort [13]. This competition ran alongside the later LWC competition, and ran four rounds over eight years with 82 initial submissions, 69 of which advanced to round one. In 2022

NIST announced the four finalists: CRYSTALS-Kyber, CRYSTALS-Dilithium, SPHINCS+, and FALCON. The first 3 of these had their standards finalized: FIPS 203 (ML-KEM) [14], FIPS 204 (ML-DSA) [15], and FIPS 205 (SLH-DSA) [16]. FALCON, designated FN-DSA and slated for publication as FIPS 206, is currently in the draft period of its standardization at the time of writing, expected to be finalized in late 2026 or early 2027. ML-KEM is a key encapsulation mechanism which is used to establish a shared secret, followed by symmetric encryption to handle data. The ML-KEM family is comprised of three parameter sets of increasing security and decreasing performance efficiency: ML-KEM-512, ML-KEM-768, ML-KEM-1024, with ML-KEM-512 being the most constrained friendly variant, though still imposing a significant overhead on small microcontroller targets [27, 28]. In March of 2025 NIST also selected HQC as a backup KEM to ML-KEM, acting as a mathematically distinct alternative that was based on error-correcting codes rather than lattices [29]. As its draft standard is not expected to arrive until 2026 with finalization in 2027, HQC falls outside of the scope of this study, due to the goal of evaluating algorithms against their published standards.

ML-KEM was designed with security in mind, not for devices running in constrained environments [28]. ML-KEM is RAM intensive by nature, as it relies on lattice-based operations that involve polynomial arithmetic, specifically the Number Theoretic Transform [14]. ML-KEM-512 has rather large public key and ciphertext sizes, 800 and 768 bytes respectively, and on a device with just 10 KiB of RAM, these alone will consume a large portion of available memory [14]. When we bring these PQC algorithms over Cortex-M3 or M0 devices, RAM limitations become the main blocker, with some PQC algorithms simply not fitting due to RAM consumption, irrespective of their other properties [28]. On top of this, at the time of writing, no Class 1 or Class 2 microcontrollers include NTT hardware acceleration, meaning that all computation must be completed in software. Even heavily optimized ver-

sions of Kyber-512 designed for Cortex-M0 require millions of clock cycles per operation [27], and on a device running at 48 MHz with a coin cell battery as the budget, this has viability implications that cannot be ignored. The pqm4 paper covers Cortex-M4 only and provides no energy measurement, so the question of whether ML-KEM-512 is viable on Cortex-M0+ or RISC-V under real power constraints remains unanswered [17].

## 2.5 Summary of Prior Work & Gaps

FELICS was published as the first consistent open-source methodology designed for comparing lightweight block ciphers on embedded hardware, centering on AVR, MSP430, and ARM Cortex-M3 [30]. It demonstrated that a standardized benchmark is feasible and valuable for the wider constrained environment research community. The NIST LWC evaluation process in itself also produced substantial performance data across a very wide range of algorithms allowing for the public to effectively compare features like cycle counts and code size throughout the competition rounds [21]. Renner et al. published in 2022, represents the most thorough benchmarking completed on the NIST LWC finalists to date, evaluating the ten finalist candidates over five platforms and four architectures [25]. Finally, pqm4 provides a credible and reproducible baseline for post-quantum algorithm performance on ARM Cortex-M4, providing cycle counts and stack usage, and has been widely cited and built upon [17].

There are currently no studies that evaluate LWC and PQC algorithms in the same test harness under identical conditions. FELICS and Renner act as the closest existing benchmarking work though they cover only LWC, and pqm4 covers only PQC, all of which use incompatible methodologies, meaning that their results cannot be directly compared [17, 25, 30]. No work currently is evaluating Ascon-128, Ascon-80pq, GIFT-COFB, AES-128-GCM, and

ML-KEM-512 together on the same hardware, and the most comprehensive LWC research, Renner et al., contains no PQC primitives whatsoever [25], and on the other side pqm4 contains no classical AEAD baseline in their research [17]. The two most relevant prior studies are greatly complementary in their coverage, though there has not been a study that combines them. Across all three of these studies, they report cycle counts and code size but energy per operation from physical hardware instrumentation is absent from all of them [17, 25, 30]. This metric is crucial in IoT deployment; designers cannot make power budget decisions without this important data, yet it does not exist in literature for this combination of algorithms and platforms. Most of today’s widely deployed IoT architectures are also the least studied in existing literature: pqm4 is Cortex-M4 only [17], FELICS looks at older architectures like AVR and MSP430 [30], and Renner does not include Cortex-M0+ or RISC-V [25]. Metrics such as security bits per cycle and security bits per microjoule have not been derived across this algorithm and platform either, metrics that this research derives and presents. Currently there is no study that addresses all of these gaps simultaneously, in this study, we address the issue of unified methodology, physical energy measurement, cross-family algorithm coverage, and cross-architecture platform coverage, directly addressing the gaps identified in the research questions of this study.

This thesis addresses all three gaps simultaneously: unified methodology, cross-family comparison, and physical energy measurement. ORBIT provides an open-source framework that can be applied across all five target platforms, enabling direct comparison. The target platforms that have been selected: RP2040 (Cortex-M0+), nRF52840 DK (Cortex-M4), STM32 Nucleo (Cortex-M4), ESP32-c61 (RISC-V), and Raspberry Pi 5 as a reference, represent the most commonly deployed constrained hardware classes in current IoT deployments [18]. Beyond raw performance data, the derived security-efficiency metrics provide a novel contribution that allows for raw benchmark data to be converted into actionable design guidance.



The resulting dataset gives IoT designers the foundation to make informed decisions when selecting cryptographic primitives for both current deployments and post-quantum transition planning [\[12\]](#).

# Chapter 3

## Methodology

### 3.1 Experimental Overview

This chapter describes the methodology used to evaluate lightweight and post-quantum cryptographic algorithms across the selected hardware platforms. The methodology is built around ORBIT, a portable benchmarking framework that separates algorithm implementations from platform-specific timing, trigger, and output code. Each algorithm-platform combination is evaluated using deterministic inputs, fixed benchmark parameters, correctness validation, repeated runs, and a unified CSV output schema. Performance, memory, and energy results are then derived from the collected benchmark output and external power traces.

### 3.2 Algorithms Under Evaluation

The algorithms in this study span classical AEAD, lightweight AEAD, and post-quantum key encapsulation. They were selected to compare a widely deployed software baseline, NIST’s standardized lightweight AEAD, a lightweight finalist with a different internal structure, a symmetric post-quantum hedge, and a standardized post-quantum KEM.

AES-128-GCM acts as the baseline primarily because it represents the industry standard

when it comes to security on general purpose systems [6]. Though one of the most important reasons that it is the baseline here is because it is not designed with constrained systems in mind, our goal is to compare the incumbent standard against much more light and efficient algorithms that will perform faster on constrained systems. AES-GCM has a 128-bit key, 96-bit nonce, and a 16-byte tag, and has GCM mode enabled on top of standard AES which provides AEAD in a single pass [6].

Ascon-128 was selected as it is the NIST SP 800-232 primary AEAD standard [22], a result of a public competition spanning six years [21], and is the primary algorithm this study is motivated to evaluate on constrained hardware. Like AES it has a 128-bit key, 128-bit nonce, and a 16-byte tag as its foundation. It uses a sponge-based Ascon-p permutation on a 320-bit state, has no lookup tables, and its state fits in the register file of a 32-bit MCU [10]. Given the algorithm’s constant-time-friendly structure, it should also perform well in the timing consistency analysis.

Ascon-80pq is included as a symmetric post-quantum hedge within the Ascon family. It uses a 160-bit key and is intended to provide an 80-bit security margin against quantum brute-force search under the usual Grover’s algorithm model [23]. Compared to ML-KEM, Ascon-80pq does not solve the issue of public-key key-establishment; it is evaluated here to quantify the cost of increasing the symmetric-key security margin within a lightweight AEAD family. In the implementation used in this study, Ascon-80pq also differs from Ascon-AEAD128 in rate, so performance differences should not be attributed solely to key length [10].

GIFT-COFB is included here given its differing design philosophy from Ascon: where Ascon uses a sponge-permutation, GIFT-COFB functions using a substitution-permutation network. Comparing these two design approaches under identical conditions is a key contribution of this study. GIFT-COFB was a finalist in the same competition that Ascon was selected in [21], and is built on top of the GIFT-128 block cipher [24]. Adding COFB mode

to GIFT-128, wraps it into AEAD, using one block cipher call per input block, which makes it inverse-free during decryption [24]. GIFT-COFB also benefits from an extremely compact hardware footprint, since the bit-permutation layer becomes relatively expensive when run purely in software. Because of this we expect to see performance characteristics that diverge from Ascon, especially on software-only platforms.

ML-KEM-512 is functionally different from the AEAD algorithms we are benchmarking, and therefore it is also evaluated differently. Instead of measuring encryption throughput across varying message sizes, we measure the three operations independently: KeyGen, Encapsulate, and Decapsulate. Each operation is executed for a fixed iteration count and reported using cycles per operation and time per operation. ML-KEM is standardized in FIPS 203, and has an 800-byte public key, and a 768-byte ciphertext at the 512 level [14]. The constrained platforms that we are testing this on do not come with NTT acceleration, meaning that all computation is explicitly software [17]. ML-KEM is selected as the most constrained friendly parameter set and therefore the most viable candidate for the hardware under evaluation [14].

All of the algorithms benchmarked in this study are evaluated using publicly available reference implementations in order to ensure that the results reflect the inherent characteristics of each algorithm rather than implementation-specific optimizations. Ascon-128 and Ascon-80pq are obtained from the official Ascon team reference implementation [10], GIFT-COFB from the NIST LWC finalist submission package [24], AES-128-GCM from a portable C reference implementation targeting software-only deployment, and ML-KEM-512 from the PQClean reference-style implementation aligned with FIPS 203. Hand-optimized assembly implementations do exist for several of these algorithms, such as ML-KEM on ARM Cortex-M4 via the pqm4 framework [17], and would be expected to run with substantially lower cycle counts when compared to the reference implementations evaluated here. Comparing

reference and optimized implementations on identical hardware is identified as a direction of future work.

### 3.3 Hardware Platforms

Table 3.1: Hardware platforms evaluated in this study.

| Platform               | MCU/SoC     | ISA        | Clock   | Memory                 |
|------------------------|-------------|------------|---------|------------------------|
| Raspberry Pi Pico      | RP2040      | Cortex-M0+ | 125 MHz | 264 KB SRAM            |
| nRF52840 DK (PCA10056) | nRF52840    | Cortex-M4F | 64 MHz  | 256 KB RAM, 1 MB Flash |
| STM32 Nucleo F446RE    | STM32F446RE | Cortex-M4  | 180 MHz | 128 KB SRAM            |
| ESP32-C61              | ESP32-C61   | RV32IMAC   | 160 MHz | 512 KB SRAM            |
| Raspberry Pi 5         | BCM2712     | Cortex-A76 | 2.4 GHz | 4 GB RAM               |

The Raspberry Pi Pico represents the lowest-resource microcontroller target, while the nRF52840 DK and STM32 Nucleo provide two Cortex-M-class comparison points with different clock rates and memory capacities. The ESP32-C61 provides the RISC-V target, and the Raspberry Pi 5 is included as a gateway-class reference platform rather than a constrained IoT device. Together, these platforms span a constrained to capable range while covering both ARM Cortex-M and RISC-V embedded targets.

The Raspberry Pi Pico (RP2040) is the most constrained target in this study, and it represents the lower bound of feasibility. The Pico has a dual-core ARM Cortex-M0+ processor which runs at 125 MHz, with 264 KB of SRAM and 2 MB of flash [31]. The board does not have any hardware floating point capabilities, and does not have any hardware acceleration for AES or NTT. The M0+ lacks the DSP extensions and barrel shifter of the M4 [32], making it a valuable stress test for software-only cryptographic implementations. While the

Pico has two cores, we only make use of one of them, in order to ensure single-threaded benchmark integrity.

Compared to the Pico, the Nordic nRF52840 DK uses an ARM Cortex-M4F architecture with an FPU, operates at 64 MHz, and has 256 KB of RAM and 1 MB of flash [33]. This board also includes onboard cryptographic hardware, but for all benchmarks, we have disabled hardware AES acceleration so that results reflect software-only implementation performance [33], while remaining directly comparable across platforms. Without ensuring this, the AES-128-GCM results on the nRF52 would not be comparable to the results from the Pico, undermining the cross-architecture analysis. The M4F’s DSP extensions and FPU remain in use as they are a part of the standard ISA that software implementations can use [34].

We also use the STM32 Nucleo board, which runs on ARM Cortex-M4 as well with a clock speed of 180 MHz, 128 KB of SRAM, and 512 KB of flash [35]. Although it has the same ISA as the nRF52840, it has a different clock frequency and significantly more RAM, which allows us to analyze how microarchitectural configuration affects performance within a fixed ISA. The STM32 Nucleo does not come with any hardware acceleration built in, for AES or NTT, though like the nRF52 it benefits from DSP extensions and FPU. The ARM Cortex-M4f was the most widely adopted evaluation platform throughout the NIST LWC standardization process, and was used by five out of the six independent evaluation labs [21], making the STM32 Nucleo and the Nordic nRF52840 results the most directly comparable to prior literature [25].

The ESP32-C61 acts as the RISC-V representative and is the only non-ARM platform in the study, which allows us to separate architecture effects from algorithm effects. The C61 runs on a RISC-V RV32IMAC at 160 MHz, with 512 KB of SRAM, with 4 MB of onboard flash [36]. It is also built with WiFi and BLE capability, though radio is disabled during

benchmarking to ensure that there is no interference with power measurements. While the C61 does have AES acceleration available [36], it too is disabled in favor of a portable software implementation, and like all other boards, it is also missing NTT acceleration.

Lastly, the Raspberry Pi 5 acts as our high-capability reference platform, and is not representative of a constrained IoT deployment. In this case it is purely used to establish an upper bound of performance and to anchor our suitability matrix. It runs on an ARM Cortex-A76, at 2.4 GHz, with 4 GB of LPDDR5 memory, and boots full Linux [37]. Given our access to full Linux on this system we have opted to use standard `clock_gettime` for timing compared to bare-metal cycle results with the other boards. Results from this system represent what a gateway device or edge server could achieve given the same software implementations.

Throughout our benchmarking all of the platforms run purely on software only cryptographic implementations, with boards hardware acceleration being explicitly disabled where it is available. All of the boards are tested at their factory clock frequencies, and will not benefit from any overclocking. Systems like WiFi and BLE are also disabled in order to reduce interference with power measurements. All boards are run using single-threaded execution only regardless of their available core count. However, any features, such as DSP extensions and FPU, will remain available on M4F platforms as they are standard ISA features that are accessible to any software implementation. All of our ARM targets are compiled with ARM GCC, and the ESP32-c61 with the RISC-V GCC toolchain provided by ESP-IDF, with all implementations compiled at the `-O2` optimization level. This ensures that compiler family and optimization level are held constant within each ISA, maintaining consistency with prior benchmarking literature [17, 25]. Implementing these controls ensures that observed performance differences are attributable to ISA and microarchitectural characteristics rather than asymmetries in hardware acceleration or system configuration.

## 3.4 ORBIT Benchmarking Framework

Existing benchmarking tools do not commonly provide analysis of both AEAD and KEM algorithms across multiple bare-metal targets under identical conditions with a unified output schema. ORBIT (Open-source Reference Benchmark for IoT Targets) has been designed in order to fill this gap. ORBIT is developed with three core principles in mind: portability, modularity, and reproducibility. Its portability comes from the fact that it is developed using primarily standard C with no operating-system dependency on bare-metal targets. ORBIT was designed with modularity in mind, meaning new algorithms can be integrated by implementing the shared benchmark interface. Finally, for reproducibility, we are using deterministic test vectors, controlled iteration counts, and a unified structured CSV output. All source code used to build ORBIT is publicly available and will be archived upon completion of the thesis.

### 3.4.1 Framework Architecture

At the top level, ORBIT is organized into **bench/**, **algorithms/**, **platforms/**, and host-side tooling. The **bench/** directory houses the core benchmarking logic including message generation, measurement loops, validation checks and CSV output. The **algorithms/** directory contains the cryptographic implementations evaluated in this study. The **platforms/** directory contains board specific startup code, timing support, GPIO trigger support, and serial output functions. Host-side orchestration is provided by `orbit.py`.

A core design decision for ORBIT is that algorithm code and platform code do not directly depend on one another. The implementations of each algorithm expose a common cryptographic interface and each platform implements a small set of functions required by the benchmark core. Utilizing this separation allows the same benchmark logic to be run across



different boards and allows for new algorithms or platforms to be added without rewriting the rest of the framework.

### 3.4.2 Platform Abstraction Layer

The platform abstraction layer includes the platform-specific functions needed by the benchmarking core. These include platform initialization, cycle counter initialization, cycle counter reading, serial printing, timestamp creation, and GPIO trigger handling. The GPIO trigger functions help define measurement periods for obtaining external energy samples, as discussed in Section 3.8.

Time sources are platform specific. In platforms based on Cortex-M processors, which support the Data Watchpoint and Trace Cycle counter mechanism, the cycle counter is implemented using the DWT CYCCNT register when available [34]. In platforms on RISC-V-based architecture, the architectural cycle counter CSR is utilized [38]. In platforms without DWT capability, platform-specific time functions are used. The Raspberry Pi 5 makes use of a timing source compatible with Linux systems.

### 3.4.3 Host-Side Orchestration

The `orbit.py` script on the host manages build, flashing, execution, serial capture, and result collection tasks for the workflow. For microcontroller targets, the script calls the relevant toolchain and flasher based on the chosen board. When targeting the Raspberry Pi 5, the benchmark is compiled and run directly. At each runtime, serial output from the target is collected and stored in a separate CSV file per board and algorithm.

### 3.4.4 Benchmark Output Schema

Every benchmark test generates a well-structured row in a CSV, containing the platform identifier, algorithm name, implementation type, compiler information, clock frequency, message length, associated-data length, key length, nonce length, tag length, iteration count, run index, total encryption and decryption cycles, cycles per byte, elapsed time, memory fields, energy fields, correctness status, and notes. The energy variables are defined during the benchmark output and process, but are filled in the post-processing phase after external power traces are aligned with the benchmark windows.

## 3.5 Benchmark Protocol

The benchmark protocol is designed to keep input generation, measurement windows, and output formatting consistent across platforms. AEAD algorithms and ML-KEM-512 are evaluated using separate procedures because AEAD encryption scales with message length, while KEM operations are fixed-function public-key operations.

### 3.5.1 AEAD Benchmark Procedure

For AEAD algorithms, ORBIT evaluates encryption and decryption separately across a fixed set of plaintext lengths. The benchmark uses deterministic plaintext, associated data, key, and nonce buffers so that each platform executes the same workload. The associated-data length is fixed at 32 bytes. For each message length, ORBIT first performs a warmup phase whose results are discarded, then records the measured encryption and decryption loops.

The evaluated measurement sizes are 16, 64, 256, 1024, 4096, and 16384 bytes, where supported by the target configuration. The lightweight AEAD algorithms use the standard

ORBIT iteration schedule: 1000 iterations for 16, 64, and 256-byte messages; 500 iterations for 1024-byte messages; 200 iterations for 4096-byte messages; and 100 iterations for 16384-byte messages. AES-128-GCM is substantially slower in the portable software implementation used in this study, so a reduced iteration schedule is used to keep collection time practical: 50 iterations for 16-byte messages, 20 for 64-byte messages, 10 for 256-byte messages, 5 for 1024-byte messages, 3 for 4096-byte messages, and 2 for 16384-byte messages. Because all reported metrics are normalized by iteration count, AES-GCM remains comparable on a per-operation and per-byte basis.

### 3.5.2 ML-KEM Benchmark Procedure

ML-KEM-512 is evaluated differently from the AEAD algorithms as it does not encrypt variable-length messages directly. Instead, ORBIT measures the three KEM operations independently: KeyGen, Encapsulate, and Decapsulate. Each operation is reported as cycles per operation and time per operation rather than cycles per byte.

Each ML-KEM operation is executed for a fixed number of iterations inside a measured window. The resulting CSV row identifies the operation type in the notes field. This allows KeyGen, Encapsulate, and Decapsulate to be analyzed separately in the results chapter.

### 3.5.3 Correctness Validation

Before any timing measurements begin, ORBIT runs a known-answer test for each algorithm, where a known plaintext and key pair are encrypted and the resulting ciphertext and authentication tag are compared against a precomputed reference. This practice is borrowed from NIST’s own KAT methodology [21]. If this test fails, the run is aborted and the correctness flag in the output CSV is set to indicate failure, and no timing data is recorded for

that configuration. This ensures that all of the performance results in the dataset come from a verified correct implementation. This is important given that we are running the same source code across five different platforms using varying toolchains, word sizes, and memory alignments. Without the KAT check an implementation could produce incorrect output due to an alignment difference and still complete successfully, silently corrupting the dataset.

### 3.5.4 Independent Runs

Five independent runs are conducted for each configuration, with each run recorded as a separate row in the output CSV. Aggregate statistics such as mean, standard deviation, minimum, maximum, and 95% confidence interval are computed in post-processing from the five runs, allowing per-run variance to be analyzed independently.

## 3.6 Performance Measurement Method

Performance figures are derived from the cycle counts recorded via ORBIT during each measured window. The raw cycle totals are retained in the CSV file, while normalized figures are calculated to aid comparison of performance across different message sizes, algorithms, and platforms.

### 3.6.1 Cycle Counting

At the start of each measured operation window, ORBIT reads the platform cycle counter and stores the initial value. After the measured loop completes, ORBIT reads the counter again and computes the difference between the current and initial values. Encryption and decryption are measured separately for AEAD algorithms, while KeyGen, Encapsulate and

Decapsulate are measured separately for ML-KEM-512.

Due to the fact that each platform exposes timing differently the cycle counter implementation is provided by the platform abstraction layer. The benchmark core uses the same cycle-counting interface on every target, while each platform maps that interface to the appropriate hardware or operating-system timing source.

### 3.6.2 Cycles per Byte and Cycles per Operation

For AEAD algorithms, the primary normalized metric is cycles per byte. This is computed by dividing the total measured cycles by the product of the message length and iteration count:

$$\text{cycles/byte} = \frac{\text{total cycles}}{\text{message length} \times \text{iterations}}$$

This metric normalizes for message size and allows for algorithm efficiency to be compared across platforms and input lengths [17, 25].

In the case of ML-KEM-512 we look at cycles per operation [17] independently for KeyGen, Encapsulate, and Decapsulate, rather than cycles per byte, as KEM operations are not dependent on message size. Cycles per operation is derived as follows:

$$\text{cycles/op} = \frac{\text{total cycles}}{\text{iterations}}$$

### 3.6.3 Latency and Throughput

The latency is reported as the absolute elapsed time per operation in microseconds. For bare-metal targets, elapsed time is derived from the measured cycle count and the platform

clock frequency:

$$\text{time/op} = \frac{\text{total cycles}}{f_{CPU} \times \text{iterations}}$$

For AEAD, throughput is derived by dividing message size by the elapsed time per operation.

$$\text{throughput} = \frac{\text{message length}}{\text{time/op}}$$

Throughput is particularly important in a practical deployment assessment, as designers need to know if an algorithm can keep up with their required data rate, while latency is more important in real-time or request/response workloads where time for a single cryptographic operation matters.

### 3.7 Memory Measurement Method

Memory consumption analysis is carried out using the build files created for each platform-algorithm combination. Flash memory consumption is determined by the size of the build file or toolchain output, and static RAM usage is derived from the linker output and map-file data when available. These values are recorded in the ORBIT CSV output as platform-level memory fields for each benchmark configuration.

The flash memory consumption represents the amount of code and constant data needed by the benchmark firmware and the chosen cryptography implementation. The RAM memory consumption represents the amount of statically allocated data and writable data needed by the benchmark firmware.

## 3.8 Energy Measurement Method

Energy consumption is measured using an external shunt-based measurement setup. This approach records the voltage drop across a known series resistance and computes current and energy from the captured waveform. Compared to the cycle-counting measurements, which are collected directly by ORBIT firmware, energy values are populated during post-processing after the benchmark CSV is aligned with the external WaveForms trace.

### 3.8.1 Measurement Hardware

Energy measurements were collected using a shunt based setup with the Analog Discovery 3. For each energy-measurement target, a known resistor was placed in series with the instrumented supply rail, and the AD3 records the voltage across the shunt using Channel 1 configured as a differential measurement. At the same time, the AD3 recorded digital GPIO trigger signals that are emitted by the benchmark firmware in order to identify benchmark frame boundaries and measurement windows. The measured shunt voltage was then used to derive instantaneous current, and the known target supply voltage is used to compute instantaneous power during post-processing.

This method measures energy on the instrumented supply rail rather than isolated CPU-core energy. As a result, the measured values may include current consumed by other board-level components powered by the same rail. This limitation is addressed in Section [3.11](#).

### 3.8.2 GPIO Trigger Windowing

ORBIT utilizes GPIO trigger signals to identify the portion of the waveform corresponding to each measured cryptographic operation. At the beginning of a measured window, the

firmware asserts a trigger pin; at the end of the window it clears that pin. The AD3 records this digital signal alongside the analog shunt-voltage waveform.

The measurement boundary varied slightly by platform. On the STM32 and ESP32-C61 targets, energy was measured using the board’s instrumented supply path. On the Pico, energy was measured at the *VSYS* input rail using an external 5 V supply path provided by the AD3, so in this case the recorded values also include some of the regulator and other board-level overhead. The Nordic nRF52840 DK was measured on the VDD measurement path using the boards dedicated current measurement connector. The Raspberry Pi 5 was included in timing measurements only as a reference and was not instrumented for energy capture.

Due to the fact that this method measures current at an instrumented board supply rail instead of isolated MCU-core current, the resulting values may include consumption from other board-level components powered by the same rail. These measurements should be interpreted as board-level energy rather than pure core-only energy. This is addressed further in Section [3.11](#).

### 3.8.3 WaveForms Trace Processing

WaveForms exports the captured analog and digital samples to a CSV file. The ORBIT energy post-processing script, `process_energy.py`, loads this trace together with the corresponding ORBIT benchmark CSV. The script identifies trigger windows from the digital column, extracts the analog samples inside each window, and assigns the resulting energy measurements to the matching benchmark rows.

For AEAD rows, the post-processor expects separate encryption and decryption windows. For ML-KEM rows, the post-processor assigns one measured window to each operation row.



The populated output CSV contains total energy, average power, and energy-per-byte fields where applicable.

### 3.8.4 Energy Computation

For each trigger window, the current is computed from the measured shunt voltage:

$$I(t) = \frac{|V_{shunt}(t)|}{R_{shunt}}$$

Instantaneous power is then computed using the target supply voltage:

$$P(t) = I(t)V_{DD}$$

Total energy is calculated by numerically integrating instantaneous power over the trigger interval:

$$E = \int_{t_0}^{t_1} P(t) dt$$

For AEAD algorithms, energy per byte is computed by dividing the total measured energy by the product of message length and iteration count:

$$\text{energy/byte} = \frac{E}{\text{message length} \times \text{iterations}}$$

For ML-KEM-512, energy is reported per operation because KeyGen, Encapsulate, and Decapsulate are fixed cryptographic operations rather than variable-length message encryption operations.

### 3.9 Timing Consistency Analysis

Consistency analysis of timing is used to check whether the execution time of each algorithm is stable across iterations, serving as a proxy for constant-time behavior on each platform. Coefficient of Variation (CV), defined as the ratio of the standard deviation to the mean, is computed using the execution time of each algorithm on each platform over multiple benchmarks of varying message size. The CV allows comparison across algorithms and platforms regardless of absolute cycle counts. A CV below 1% indicates consistent execution time; a CV between 1% and 5% is marginal and worth noting in the results; and a CV above 5% is flagged for investigation as potentially indicative of data-dependent timing variation. Timing variation in cryptographic implementations can indicate that they are susceptible to timing side-channel attacks, and is particularly relevant in the case of AES-128-GCM, given its dependence on lookup tables serving as a direct point of comparison against Ascon’s constant-time-friendly sponge permutation design. This analysis serves as a consistency metric only and does not constitute a formal side-channel security evaluation.

### 3.10 Statistical Analysis

For each platform algorithm and benchmark configuration, ORBIT records independent runs using a run index in the output CSV. Post-processing groups results by platform, algorithm, operation type, and message size. For each group the mean, standard deviation, minimum, and the 95% confidence interval are computed.

The 95% CI is used to summarize uncertainty between runs in the measured metrics. Results are reported separately for encryption and decryption in AEAD algorithms, and separately for KeyGen, Encapsulate, and Decapsulate in ML-KEM-512. Invalid runs, including runs

that fail correctness validation, are excluded from aggregate statistics and identified using the correctness flag in the CSV output.

Energy statistics are computed after WaveForms traces have been processed and the energy fields in the ORBIT CSV have been populated. This allows timing, memory, and energy results to be analyzed using the same grouping structure.

## 3.11 Limitations

This methodology is supposed to produce consistently comparable results across platforms, but several limitations remain. Firstly, the evaluated implementations are portable reference implementations, and not optimized assembly implementations. Therefore, one should interpret measured cycle counts as software-portable results rather than best-case performance for each platform.

Secondly, hardware cryptographic acceleration is intentionally disabled where available to keep all comparisons at the software level. This means the AES-128-GCM results do not reflect the performance that could be achieved on platforms with AES acceleration enabled. The results instead characterize the cost of portable software AES-GCM under the same conditions as the lightweight and post-quantum implementations.

Third, energy measurements are taken at the instrumented supply rail and may include board-level current draw from components besides the MCU core. The shunt resistor tolerance, AD3 sampling configuration, probe placement, and supply-voltage assumptions all contribute to measurement uncertainty.

Fourth, the Raspberry Pi 5 is included solely as a gateway-class reference platform rather than as a constrained IoT target. Because it runs Linux rather than bare-metal firmware, its

timing behavior is influenced by operating-system scheduling and is not directly comparable to that of the MCU targets. It is therefore used only for timing comparisons and is excluded from the energy measurements.

Finally, timing consistency analysis is not a formal side-channel evaluation. Low timing variation may indicate stable execution under benchmark conditions, but it does not prove resistance to timing, power, electromagnetic, or fault-based side-channel attacks.

# Chapter 4

## Experimental Procedure and Data Collection

### 4.1 Collection Workflow

The data collection process was performed using a pipeline approach that started with selection of a target platform and algorithm, construction and running the benchmark, recording of the corresponding CSV output files, external energy traces if required, and finally post-processing of raw data into final analysis data sets.

The configuration of a benchmark refers to a combination of a target platform and a selected algorithm. If an AEAD algorithm is selected, then its configuration also includes the evaluated message lengths, with encryption and decryption reported separately. Lightweight AEAD algorithms use the standard ORBIT iteration schedule, whereas AES-128-GCM uses a reduced iteration schedule because its portable software implementation has much longer execution times across various boards in this research. These iteration counts are recorded in the output CSV and used during post-processing to report timing and energy per operation and, for AEAD rows, per byte. For ML-KEM-512, the configuration is defined by the KEM operation: KeyGen, Encapsulate, or Decapsulate. All configurations include five iterations to allow variation analysis. Table [4.1](#) summarizes the major stages of this workflow.

Table 4.1: ORBIT data collection workflow.

| Stage                  | Description                                                                                               | Tool                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------|
| Build configuration    | Select the target board and algorithm, then produce the corresponding firmware image or local executable. | <code>orbit.py</code> , CMake, ESP-IDF |
| Target execution       | Flash the target board or execute the benchmark locally for the Raspberry Pi 5.                           | Board flash tools                      |
| Output capture         | Capture the benchmark’s CSV output stream and save raw rows to the results directory.                     | <code>orbit.py</code>                  |
| Energy trace capture   | Record GPIO trigger signals and the shunt-voltage waveform during benchmark execution.                    | AD3, WaveForms                         |
| Energy post-processing | Align the WaveForms trace with ORBIT rows and populate energy fields in the benchmark CSV.                | <code>process_energy.py</code>         |
| Dataset assembly       | Validate processed files, exclude incomplete runs, and compute summary statistics for analysis.           | Analysis scripts                       |

Each platform-algorithm configuration is collected over five independent runs. The run index from this is recorded in the output CSV, allowing for rows from the same configuration to be grouped and evaluated during post-processing. On top of this variation between runs is preserved during collection, ensuring that these values are not averaged away. The resultant benchmark files are then stored under `results/<board>_<algorithm>.csv`.

Runs are only validated when the criteria is satisfied. The benchmark is expected to be run completely, and the emitted CSV rows must match the expected schema with a correctness flag validating their correctness. In order to validate that the incoming data is useful, any incomplete serial captures, failed correctness checks, or missing rows are recollected. The accompanying energy values are accepted only after the WaveForms trace is post-processed and correctly aligned with the expected trigger windows. Rows with unpopulated energy fields are treated as timing-only records until energy processing is complete.

## 4.2 Host Environment and Toolchains

This system is designed to be used primarily with a Windows 11 host running Ubuntu 24.04 LTS under WSL2, though in the case of the Raspberry Pi 5 it is also designed to run natively on Linux to run the benchmark on itself. Python is used for host-side orchestration and post-processing. CMake and platform-specific toolchains are used for firmware builds and flashing. Board-specific flashing tools such as ESP-IDF and OpenOCD are used to flash targets for benchmark execution. Table 4.2 goes into the specific host environment and software needed in order to run these benchmarks.

Table 4.2: Common host environment and software used for data collection.

| Component                   | Purpose                                                                                                                   | Used for                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| Windows 11                  | Host operating system used for USB device access and WaveForms instrumentation.                                           | All USB-attached targets and AD3 capture |
| Ubuntu 24.04 LTS under WSL2 | Linux environment used for ORBIT development, firmware builds, flashing workflows, serial capture, and result processing. | Pico, STM32, nRF52840, ESP32-C61         |
| usbipd-win                  | Forwards USB devices from Windows into WSL2 so flashing and serial capture tools can access target boards.                | Pico, STM32, nRF52840, ESP32-C61         |
| Python 3.12+                | Runs ORBIT orchestration, serial capture, CSV handling, energy post-processing, and plotting scripts.                     | All targets                              |
| pyserial                    | Captures UART/virtual COM benchmark output from microcontroller targets.                                                  | Pico, STM32, nRF52840, ESP32-C61         |
| pandas and matplotlib       | Support post-processing, aggregation, and result visualization.                                                           | Dataset analysis and plotting            |
| CMake                       | Configures and generates builds for ORBIT firmware and native targets.                                                    | Pico, STM32, nRF52840, Raspberry Pi 5    |
| Digilent WaveForms          | Records Analog Discovery 3 analog and digital traces for energy measurement.                                              | Energy trace collection                  |
| Analog Discovery 3          | Captures shunt-voltage waveforms and GPIO trigger signals.                                                                | Energy measurement                       |
| tools/orbit.py              | Builds selected configurations, flashes targets where applicable, captures benchmark output, and writes raw CSV files.    | All benchmark collection                 |
| tools/process_energy.py     | Aligns WaveForms traces with ORBIT CSV rows and populates energy fields.                                                  | Energy post-processing                   |

Compiler identifiers recorded in the ORBIT CSV output allow tool versions to be traced to

individual collected runs.

Full target-specific toolchain details are provided in [Appendix A](#)

## 4.3 Target Execution Procedures

To avoid USB device conflicts during data collection, only one target board is connected to the host machine at a time. This makes `usbipd` attachment and serial-port assignment more reliable, especially when several boards expose similar virtual COM interfaces. The benchmark and flash procedures follow a common pattern across the MCU targets: ORBIT builds the selected benchmark image, flashes or resets the target, captures the emitted CSV output, and writes the results using the `results/<board>\_<algorithm>.csv` naming convention. The Raspberry Pi 5 differs from the MCU targets because it is executed as a native Linux process rather than flashed firmware, while the ESP32-C61 uses the ESP-IDF build and flashing workflow instead of the common CMake path used by the ARM MCU targets.

### 4.3.1 Raspberry Pi Pico (RP2040)

For the Raspberry Pi Pico target, ORBIT configures the CMake build for the selected algorithm and cross-compiles the firmware using the Pico SDK and `arm-none-eabi-gcc`. The UF2 image produced is then flashed through the Pico’s BOOTSEL mass-storage workflow. For the initial run, the board is placed into BOOTSEL mode manually; for the later runs, ORBIT attempts to reboot the board into BOOTSEL using `picotool` and reattach the device to WSL2 through `usbipd`.

After flashing, ORBIT waits for the Pico serial device to become available and captures



the emitted benchmark CSV output. Captured rows are then written to `results/pico\_allowbreak<algorithm>.csv`. If automatic USB attachment or mounting fails, the board is reattached manually, and the affected run is repeated.

### 4.3.2 Nordic nRF52840

For the Nordic nRF52840 PCA10056, ORBIT configures the CMake build for the selected algorithm and cross-compiles the firmware using the nRF5 SDK and `arm-none-eabi-gcc`. The board's J-Link USB interface is then mounted to WSL2. ORBIT makes use of `nrfjprog` to erase, program, verify, and reset the board for flashing, allowing each run to start from a known reset state and be completed automatically.

Because the benchmark can begin emitting output immediately after reset, ORBIT opens the J-Link virtual COM serial port before issuing the rest used for capture. CSV rows are captured and written to `results/nrf52\_<algorithm>.csv`. If the J-Link mount or serial attachment fails, the device is remounted manually using `usbipd`, with the affected run discarded and the flashing repeated.

### 4.3.3 STM32 Nucleo F446RE

For the STM32 Nucleo F446RE, ORBIT configures the CMake build for the selected algorithm and cross-compiles the firmware using `arm-none-eabi-gcc` with STM32CubeF4 support. The board is mounted to WSL2 through `usbipd` using its ST-Link USB interface. ORBIT utilizes OpenOCD with the ST-Link interface and STM32F4 target configuration to flash the generated firmware for each run.

After flashing and reset, ORBIT captures benchmark output from the board's virtual COM

port. CSV rows are captured and written to `results/stm32\_<algorithm>.csv`. The board is subsequently reset and reflashed upon completion of each benchmark run. If multiple serial devices are visible, the capture port is specified explicitly using the `--port` option.

#### 4.3.4 ESP32-C61

The ESP32-C61 target uses a separate ESP-IDF workflow rather than the common CMake firmware path used by the ARM microcontroller targets. To start, the ESP-IDF environment is activated before collection. Using ORBIT, the ESP32 target and algorithm are selected for benchmarking. ORBIT automatically configures the ESP-IDF project based on the selected algorithm. ESP-IDF builds the firmware using the RISC-V GCC toolchain. ORBIT invokes `idf.py` to set or build the target as needed.

The board is then flashed using the ESP-IDF workflow, which uses `esptool.py` under the hood. Once flashed, ORBIT opens the ESP serial port and runs the benchmark after reset or boot. The output is captured over the serial monitor stream and written to `results/esp32c61\_<algorithm>.csv`. As before, the board is reset between runs to ensure that the benchmark starts from a known state.

#### 4.3.5 Raspberry Pi 5

The Raspberry Pi 5 target is built and executed directly on the Pi rather than flashed from the host system. After cloning ORBIT onto the Raspberry Pi 5, the native Linux dependencies required for the local benchmark path are installed, including Python, CMake, a native compiler toolchain, and the Python packages listed in `requirements.txt`. The microcontroller flashing dependencies used by the other targets are not required for the Raspberry Pi 5 workflow.

For each selected algorithm, ORBIT builds a native Linux executable and runs the benchmark as a local process. The output is captured from standard output rather than a serial interface and is written to `results/rpi5\_<algorithm>.csv` on the Pi. Each independent run is performed by launching the benchmark process again. ORBIT’s suite mode can also be used on the Raspberry Pi 5 to build and execute each selected algorithm locally, producing one result file per algorithm.

## 4.4 Energy Trace Collection

### 4.4.1 Measurement Setup

Energy trace collection was performed using a shunt-based measurement setup with the Analog Discovery 3. The analog input measures voltage drop across the shunt resistor, while digital inputs capture the trigger signals that are emitted by ORBIT firmware. Table 4.3 summarizes the representative signal connections used during energy trace collection.

Table 4.3: Analog Discovery 3 signal connections for energy trace collection.

| Signal              | AD3 connection | Purpose                                                                                                            |
|---------------------|----------------|--------------------------------------------------------------------------------------------------------------------|
| Shunt positive side | Channel 1+     | Measures the positive side of the shunt voltage drop.                                                              |
| Shunt negative side | Channel 1-     | Measures the negative side of the shunt voltage drop, allowing Channel 1 to record the differential shunt voltage. |
| Measurement trigger | DIO0           | Marks the beginning and end of each measured encryption, decryption, or KEM operation window.                      |
| Frame trigger       | DIO1           | Marks the beginning and end of a complete benchmark run during multi-run captures.                                 |
| Target ground       | AD3 ground     | Provides a shared reference for digital trigger capture and analog measurement.                                    |

The target GPIO pins connected to DIO0 and DIO1 vary by board, but their roles remain the

same throughout the measurement workflow. DIO0 is used for operation-level measurement windows, while DIO1 is used as a run-level frame marker. For the ESP32-C61 validation run, GPIO2 is connected to DIO0 and GPIO3 is connected to DIO1

#### 4.4.2 Trace capture Procedure

Once the target board has been wired to the shunt and AD3, WaveForms recording is started before the benchmark begins. ORBIT is then used to build, flash, and execute the selected benchmark configuration. During execution, the firmware asserts the frame trigger at the beginning of the benchmark run and clears it after the run completes. Within that frame, the measurement trigger is asserted around each measured cryptographic window.

For AEAD algorithms, each message size produces two measurement windows: one for encryption and one for decryption. For ML-KEM-512, each operation row produces one measurement window corresponding to KeyGen, Encapsulate, or Decapsulate. The WaveForms recording is stopped only after both the measurement trigger and the frame trigger have returned to low, ensuring that the final operation window is not truncated.

The WaveForms trace is then exported as a CSV file containing the Channel 1 shunt-voltage waveform and the digital trigger bitfield. For the current measurement setup, Channel 1 records the differential voltage across the shunt using the Channel 1+ and Channel 1- inputs. The exported trace is then paired with the matching benchmark CSV during post-processing.

The ESP32-C61 Ascon-AEAD128 test run was used to validate this capture procedure. A single AEAD run with six message sizes should produce 12 measurement windows within a single frame window. This validation confirmed that the trigger pins and two-column WaveForms export were interpreted correctly by the post-processing script. It also established an acceptance condition for later captures: the recording must include the falling edge of the

final measurement window and the falling edge of the frame trigger.

For the Raspberry Pi Pico and the Nordic nRF52840 DK, energy capture had to be performed without an active USB connection to the host. To support this workflow, ORBIT was built in an energy-measurement mode that emitted frame and measurement GPIO triggers and executed five internal runs automatically after boot. Once the capture for a given algorithm was complete, the board was reflashed to proceed to the next algorithm.

### 4.4.3 Trace Organization

Raw ORBIT CSV files are stored in `results/`, while raw WaveForms exports are stored separately in `results/traces/`. Trace filenames identify the board, algorithm, and run range so that each waveform can be paired with the correct ORBIT output file during post-processing. Processed CSV files are written with an energy-specific suffix during validation runs, so the original timing-only output remains available for comparison.

For final collection, each trace is associated with a known shunt resistance, supply voltage, sample rate, board, algorithm, and run range. Captures that do not include the expected trigger windows or that end before the frame trigger returns low are excluded and recollected. Preliminary traces are retained only for debugging and are not included in the final analysis dataset.

## 4.5 Post-processing and Dataset Assembly

In this study, a run refers to an independent benchmark execution, while an iteration refers to one repeated cryptographic operation inside a measured loop. Energy is first computed for the full measurement window and is then normalized by the recorded iteration count.

The `process\_energy.py` script takes in two files as input: the WaveForms exported CSV and the ORBIT benchmark CSV. The script automatically detects which field is the digital column and uses Channel 1 voltage as the shunt voltage signal. The frame windows, identified using DIO1, mark the beginning and end of a complete benchmark run, while the measurement windows, identified using DIO0, mark the start and end of a measurement. Each ORBIT CSV row is mapped to its corresponding trigger windows. AEAD rows expect exactly two windows per row: encryption and decryption, and ML-KEM rows expect exactly one window per row, for each of the three cryptographic operations. Using the calculations discussed in Section 3.8.4, the total energy for each measurement window is calculated first. For AEAD rows, encryption and decryption windows are stored separately as total energy over the measured loop. These values are then normalized by the iteration count to obtain energy per operation, and by the product of message length and iteration count to obtain energy per byte. For ML-KEM-512 rows, each trigger window corresponds to one KEM operation type repeated for the configured iteration count, so the measured window energy is divided by the iteration count to obtain energy per KeyGen, Encapsulate, or Decapsulate operation. Once this is complete, the processed CSV is written to the final dataset file.

## 4.6 Data Quality Checks

After post-processing, each benchmark configuration is checked for structural completeness, correctness, trigger alignment, and measurement plausibility before being included in the final analysis dataset. These checks are intended to identify missing runs, incomplete traces, failed correctness validation, or energy values that require recollection before analysis. Table 4.4 summarizes the checks applied to each processed configuration.

Table 4.4: Data quality checks applied to each processed benchmark configuration.

| Check                      | Expected Value                                                                   | Action if Failed                                                                           |
|----------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Row count                  | Matches expected count for the configuration.                                    | Identify missing rows and recollect the affected run.                                      |
| Iteration count            | Matches the configured schedule for the algorithm and message size.              | Verify that normalized metrics use the recorded iteration count before accepting the file. |
| correctness flag           | ok = 1 for all accepted rows.                                                    | Discard and recollect any run containing a failed correctness check.                       |
| trigger window count       | 2× row count for AEAD; 1× row count for ML-KEM.                                  | Recollect if windows are missing, unpaired, or orphaned.                                   |
| Final digital state        | All trigger signals return to zero at the end of the trace.                      | Treat trace as incomplete and recollect.                                                   |
| Energy fields populated    | Window energy and derived per-operation energy are nonzero after post-processing | Re-run <code>process\_energy.py</code> or recollect the energy trace.                      |
| Average power plausibility | Within physically reasonable range for the target board.                         | Inspect wiring and shunt placement before recollecting.                                    |
| Energy scaling             | Energy per byte decreases consistently with increasing size for AEAD rows.       | Flag for investigation before including in the final dataset.                              |

# Chapter 5

## Results

This chapter presents performance, energy, memory, and timing-consistency results. Cross-platform timing analysis has been performed on all five platforms, while energy plots have been measured only for the MCU platforms, with the Raspberry Pi 5 excluded as a reference gateway-class platform.

### 5.1 Cross-Platform Performance Results

#### 5.1.1 AEAD Performance

This subsection compares AEAD encryption performance across the five chosen platforms. Figures 5.1 and 5.2 use a representative message size of 1024 B. The all-platform figures, such as Figure 5.1, are used to show the big picture, whereas the MCU-only figures, such as Figure 5.3, aim to make the embedded differences more readable.

Figure 5.1 shows the broad separation between the Raspberry Pi 5 and the MCU targets. Using Ascon-128, the Pi 5 averaged 0.305 cycles/byte, while the best MCU (ESP32-C61) had an average cycles per byte of 104.8. The lightweight AEADs have proven to be significantly cheaper than software AES-GCM on every MCU, with the ESP32-C61 averaging 104.8 cycles/byte for Ascon-128, and 32,575.1 cycles/byte for AES-GCM, which is roughly  $235\times$  higher.



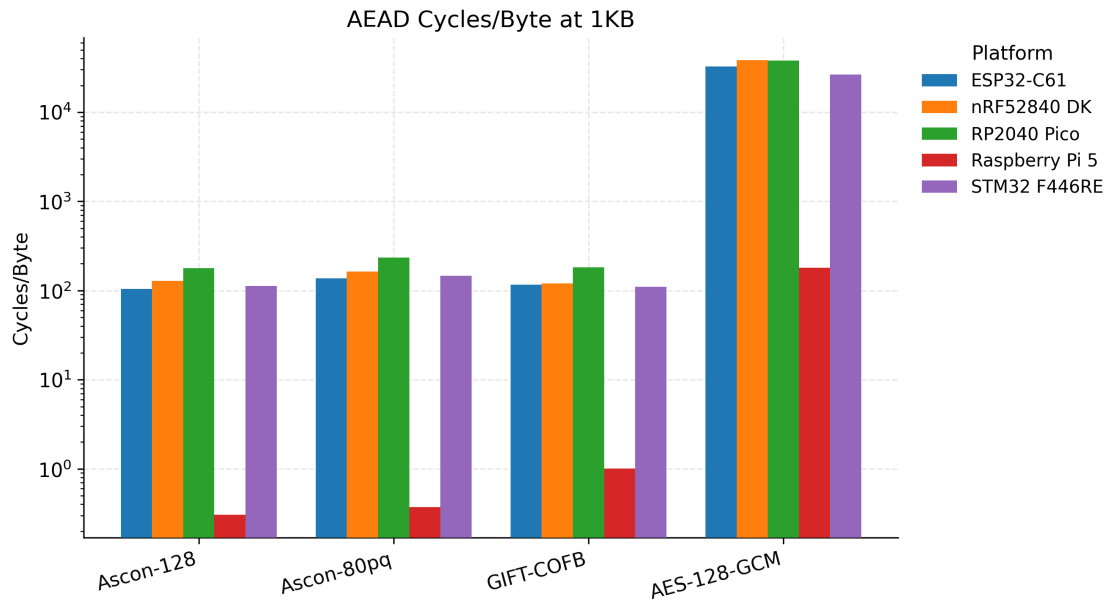


Figure 5.1: AEAD encryption cycles per byte at 1024-byte message size across all evaluated platforms.

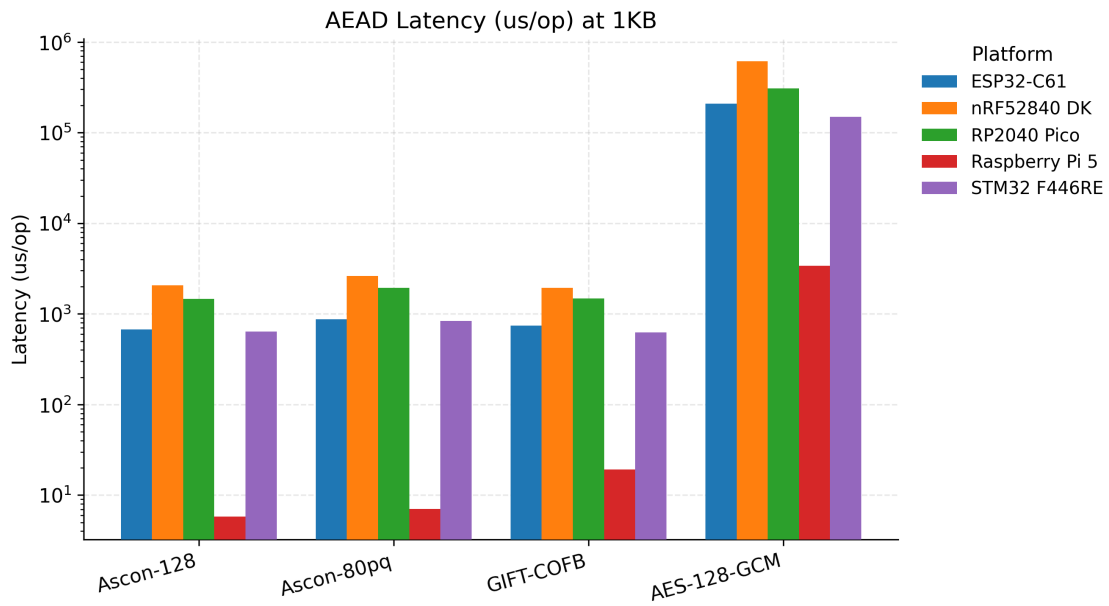


Figure 5.2: AEAD encryption latency at 1024-byte message size across all evaluated platforms.

As shown in Figure 5.2, the same story as cycles/byte is reinforced. The STM32 and ESP32-C61 are the fastest performers for AEAD workloads, with Ascon-AEAD128 latencies of 639.3  $\mu$ s and 670.6  $\mu$ s, respectively, while the nRF52840 DK and Pico lag behind at 2058.5  $\mu$ s and 1467.9  $\mu$ s. Comparing the lightweight AEADs to the classical AES-GCM, the MCU target latencies are pushed into the hundreds of milliseconds, reaching 150101.5  $\mu$ s on the STM32, 208480.5  $\mu$ s on the ESP32-C61, 309325.0  $\mu$ s on the Pico, and 612184.7  $\mu$ s on the nRF52840 DK.

Figure 5.3 is provided as an MCU-only plot, since including the Raspberry Pi 5 compresses the rest of the data due its substantially higher performance.

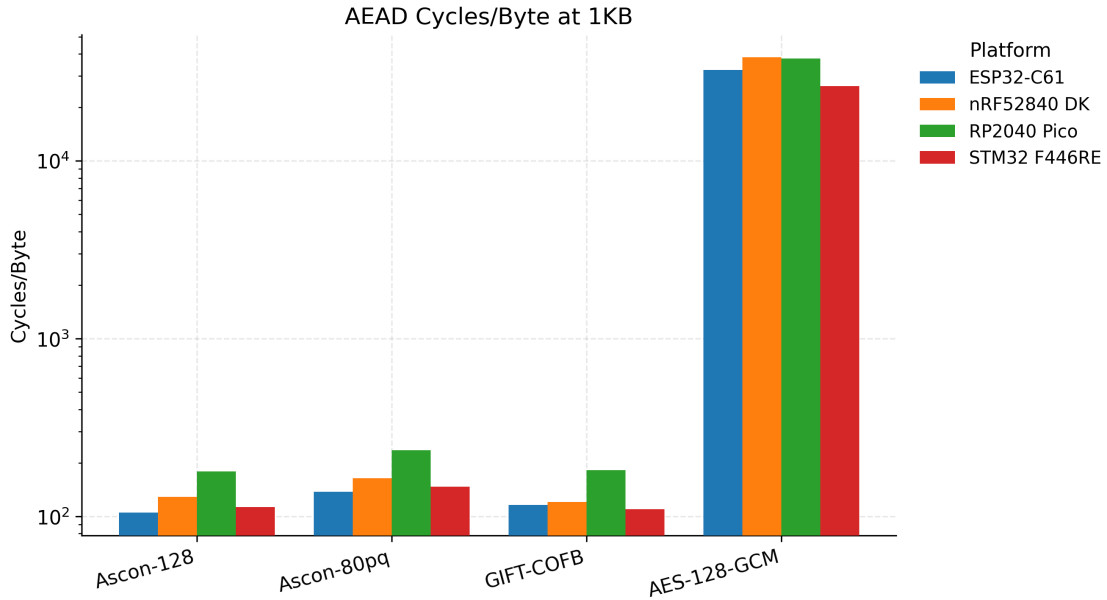


Figure 5.3: AEAD encryption cycles per byte at 1024-byte message size for MCU-class platforms only.

In Figure 5.3, the STM32 and ESP32-C61 are the strongest MCU performers. For Ascon-AEAD128 at 1024 bytes, the ESP32-C61 records 104.8 cycles per byte, while the STM32 records 112.4, compared to 128.7 on the nRF52840 and 179.2 on the Pico. GIFT-COFB remains competitive with Ascon-AEAD128 and even outperforms it on the STM32 and

nRF52 at 1024 bytes, with 109.8 versus 112.4 cycles per byte on the STM32 and 120.6 versus 128.7 on the Nordic platform. The figure also shows Ascon-80pq consistently performing slower than Ascon-AEAD128, with the overhead visible on every MCU target. Overall, the Pico is generally the slowest board among the 4 MCU targets, only narrowly beating the nRF52840 when it comes to AES-128-GCM, recording 37759.4 cycles per byte compared to 38261.5.

Figure 5.4 focuses on a single algorithm across all of the MCU targets, allowing for a more granular analysis of how the boards perform across the various message sizes.

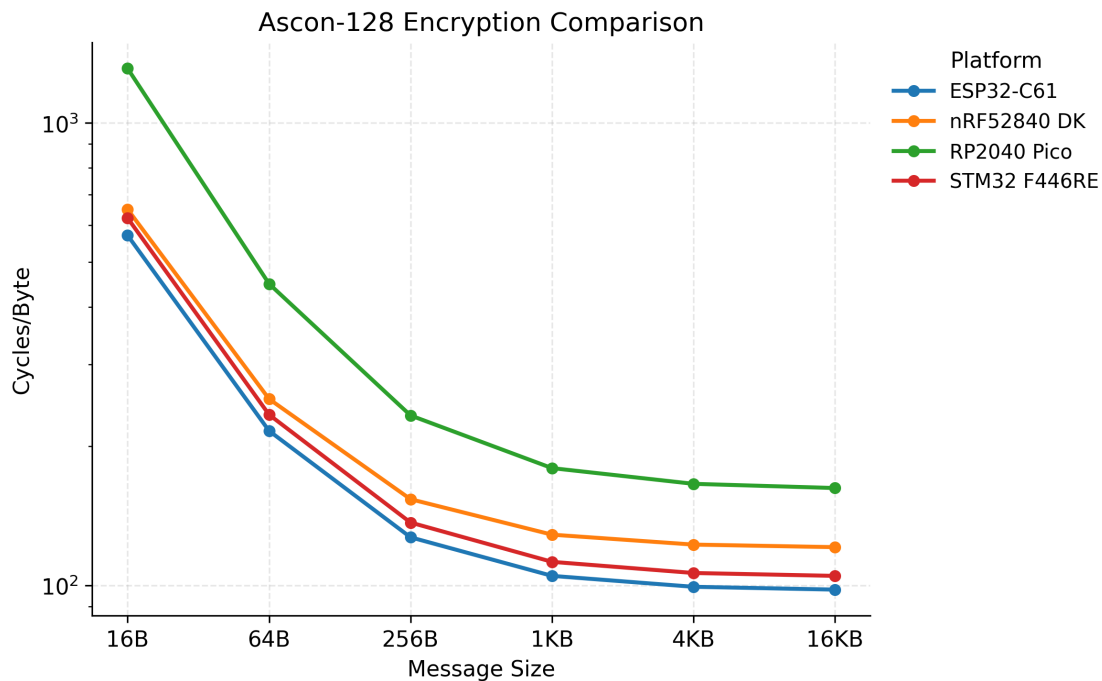


Figure 5.4: Ascon-AEAD128 encryption cycles per byte across message sizes for MCU-class platforms.

As seen in Figure 5.4, cycles per byte consistently decreases as message sizes increase for all MCU platforms. This reflects the amortization of fixed per-message cost over larger payloads. On the ESP32-C61, Ascon-AEAD128 decreases from 571.0 cycles per byte at 16 bytes to 97.9 at 16 KB, while the STM32 decreases from 622.4 to 104.8 over the same range.

The nRF52840 DK decreases from 650.2 to 120.9, and the Pico from 1310.7 to 162.3. The order of the boards remains consistent across message sizes, with the ESP32-C61 and STM32 as the strongest performers, followed by the nRF52 and the Pico the slowest.

### 5.1.2 ML-KEM-512 Performance

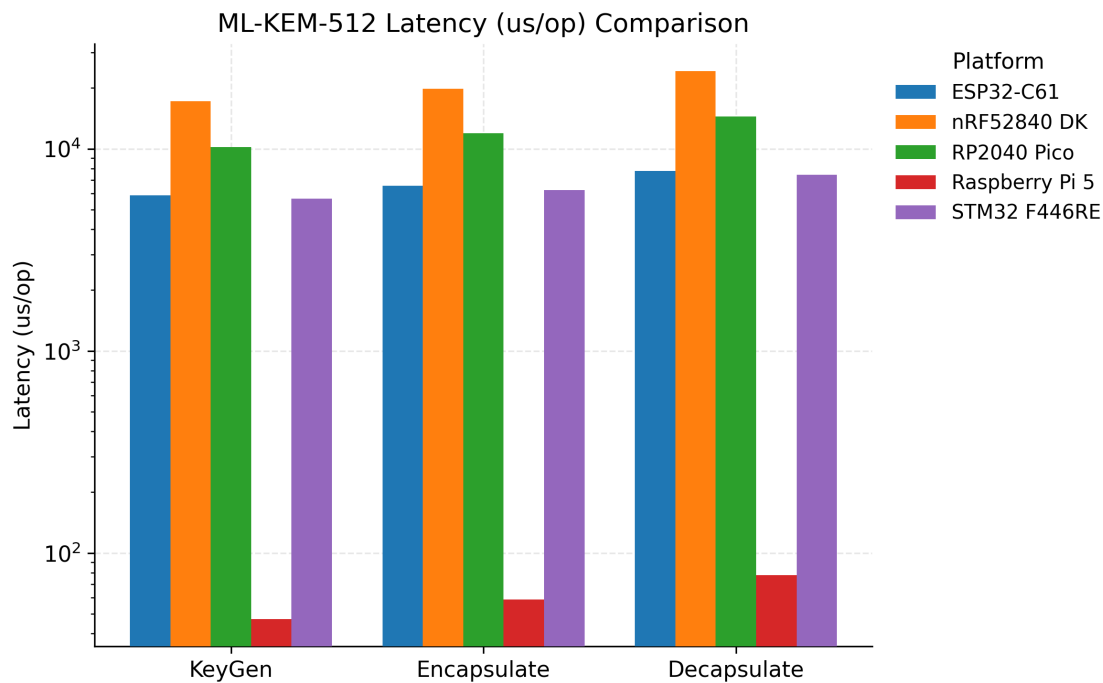


Figure 5.5: ML-KEM-512 per-operation latency across all evaluated platforms.

Figure 5.5 compares the latency of ML-KEM-512 across all of the evaluated platforms for KeyGen, Encapsulate, and Decapsulate. The Raspberry Pi 5 remains in a completely different class, completing KeyGen, Encapsulate, and Decapsulate in 47.2  $\mu$ s, 58.7  $\mu$ s, and 77.8  $\mu$ s, respectively. The STM32 and the ESP32-C61 are again the strongest performers of the MCU targets, having extremely close latencies across all three ML-KEM operations. For example, KeyGen takes 5654.0  $\mu$ s on the STM32 and 5887.0  $\mu$ s on the ESP32-C61, compared with 10209.9  $\mu$ s on the Pico and 17237.1  $\mu$ s on the nRF52840 DK. A similar pattern is observed

for both Encapsulate and Decapsulate, with the Pico slower than the STM32 and ESP32-C61, and the nRF52840 the slowest across all operations. Across all boards, Decapsulation consistently proved to be the most expensive ML-KEM operation, while KeyGen was the fastest.

## 5.2 Energy Results

### 5.2.1 AEAD Energy Efficiency

This subsection compares the energy efficiency of AEAD encryption across the four measured MCU targets. The 1024-byte message size is used again as a representative reference point for cross-platform comparison.

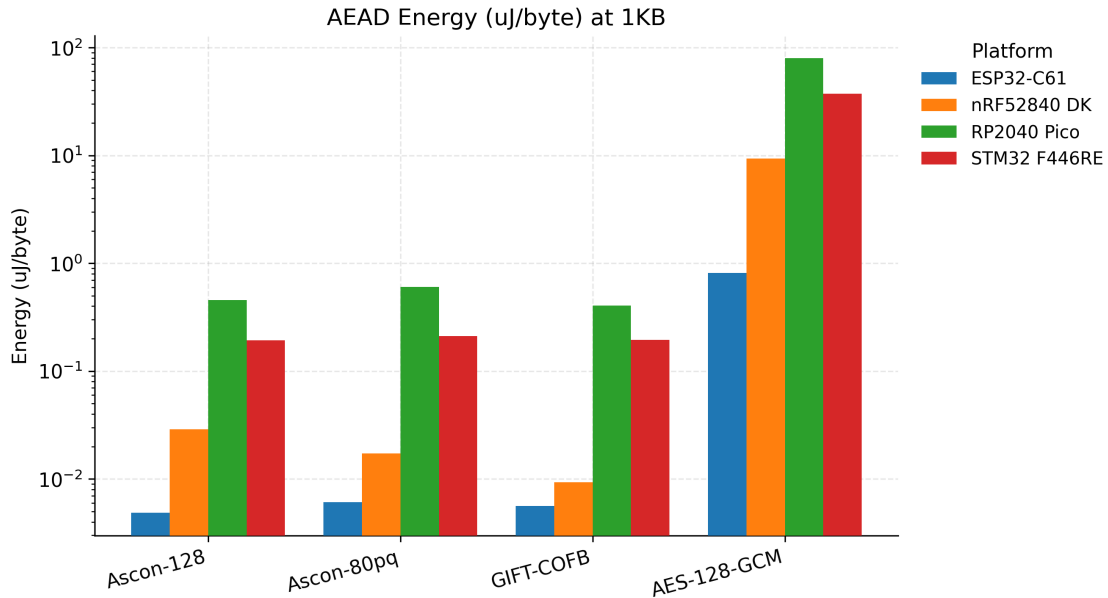


Figure 5.6: AEAD encryption energy per byte at 1024-byte message size for energy-measured embedded platforms.

Figure 5.6 shows that lightweight algorithms are dramatically more energy-efficient than

software-implemented AES-128-GCM. On the ESP32-C61, Ascon-AEAD128 requires only 0.00486  $\mu\text{J}/\text{byte}$  at 1024 bytes, compared with 0.8144  $\mu\text{J}/\text{byte}$  for AES-128-GCM. A similar gap is shown on the STM32, where Ascon-AEAD128 requires 0.1923  $\mu\text{J}/\text{byte}$  and AES-128-GCM requires 37.3488  $\mu\text{J}/\text{byte}$ . The ESP32-C61 and the nRF52840 DK prove to be especially efficient for lightweight AEADs, while the Pico remains the least efficient board across the algorithms, with Ascon-AEAD128 measured at 0.4573  $\mu\text{J}/\text{byte}$  and Ascon-80pq at 0.6060  $\mu\text{J}/\text{byte}$ . As seen with the computational cost and latency results, Ascon-AEAD128 and GIFT-COFB are quite close in energy cost. For example, on STM32, Ascon-AEAD128 and GIFT-COFB measure 0.1923  $\mu\text{J}/\text{byte}$  and 0.1940  $\mu\text{J}/\text{byte}$ , respectively. Ascon-80pq carries an overhead for most boards and performs slightly less efficiently on the ESP32, STM32, and Pico. Interestingly, the Nordic board measures both GIFT-COFB and Ascon-80pq values below Ascon-AEAD128 in energy per byte, with values of 0.00936  $\mu\text{J}/\text{byte}$  and 0.01727  $\mu\text{J}/\text{byte}$  compared with 0.02895  $\mu\text{J}/\text{byte}$  for Ascon-AEAD128.

As shown in Figure 5.7, AES-GCM energy per byte decreases substantially as message size increases across all MCU targets. On the ESP32-C61, AES-128-GCM falls from 2.6737  $\mu\text{J}/\text{byte}$  at 16 bytes to 0.6170  $\mu\text{J}/\text{byte}$  at 16 KB. The same trend appears on the STM32, where it decreases from 117.6443  $\mu\text{J}/\text{byte}$  to 22.9246  $\mu\text{J}/\text{byte}$ , and on the Pico, where it decreases from 309.8625  $\mu\text{J}/\text{byte}$  to 64.1072  $\mu\text{J}/\text{byte}$ . This again reflects the fixed setup cost being amortized over longer payloads. Despite this improvement, AES-GCM remains far more energy-intensive than the lightweight AEADs. The ESP32-C61 is by far the most energy-efficient MCU for AES-GCM across all message sizes, with its highest measured value, 2.6737  $\mu\text{J}/\text{byte}$ , still below the lowest measured value of the next-closest board, the nRF52840, at 8.4724  $\mu\text{J}/\text{byte}$ .

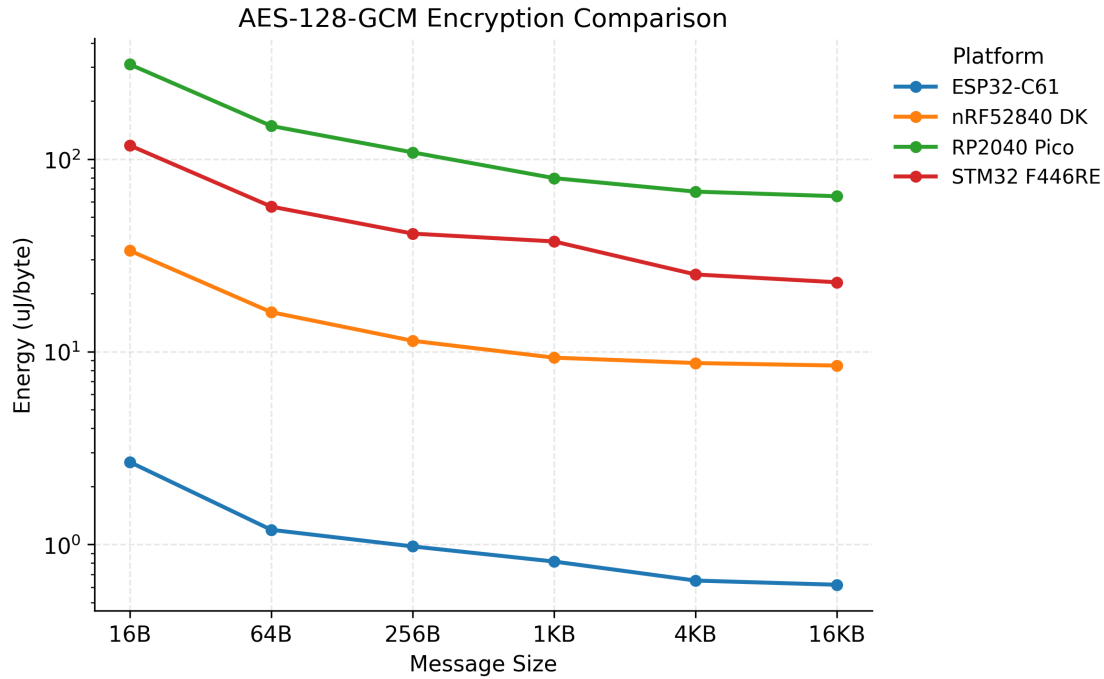


Figure 5.7: AES-128-GCM encryption energy per byte across message sizes for energy-measured embedded platforms.

### 5.2.2 ML-KEM Energy Cost

Figure 5.8 compares ML-KEM-512 energy per operation across the measured MCU platforms. The Pico proves to be the most energy-hungry board here, consistently consuming more energy than the other boards, requiring 3399.99  $\mu\text{J}$  for KeyGen, 3867.65  $\mu\text{J}$  for Encapsulate, and 4609.41  $\mu\text{J}$  for Decapsulate. The STM32 is the next most expensive board, using 962.42  $\mu\text{J}$ , 1084.54  $\mu\text{J}$ , and 1255.05  $\mu\text{J}$ , respectively. The ESP32-C61 and the nRF52840 DK are much lower energy platforms for ML-KEM-512. On the ESP32-C61, KeyGen, Encapsulate, and Decapsulate require 234.28  $\mu\text{J}$ , 259.86  $\mu\text{J}$ , and 153.60  $\mu\text{J}$ , while the nRF52840 DK measures in at 298.37  $\mu\text{J}$ , 321.44  $\mu\text{J}$ , and 247.96  $\mu\text{J}$ . On the STM32, Pico, and nRF52840, Decapsulation is the most expensive operation, whereas on the ESP32, Decapsulation is the least expensive, with Encapsulation being the most energy-intensive.

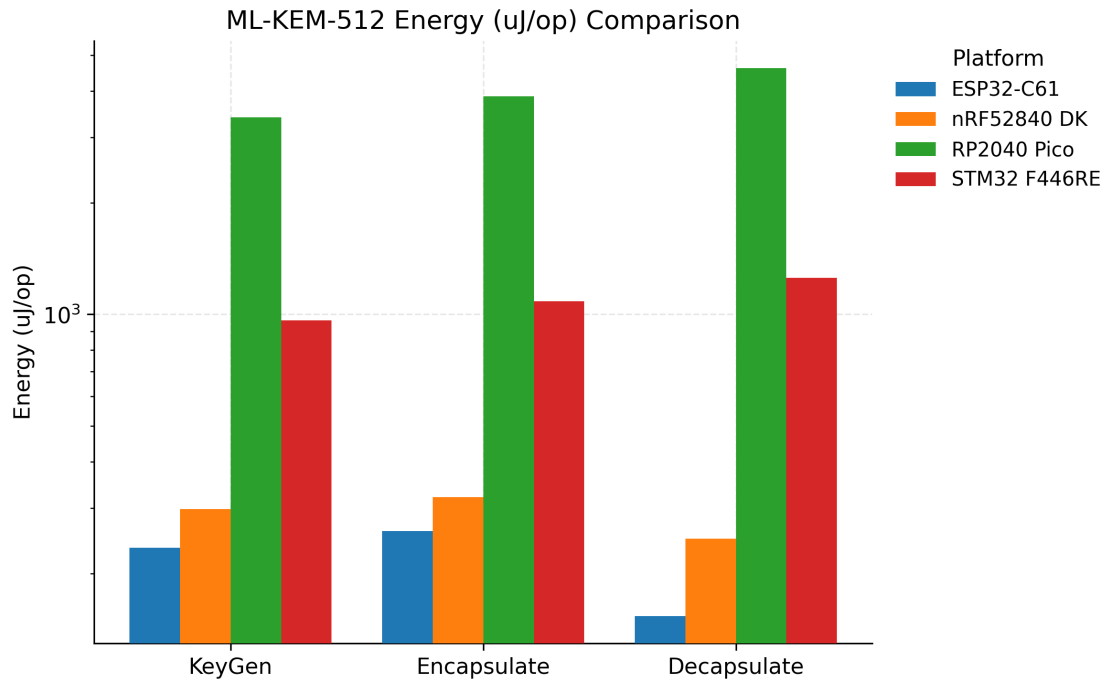


Figure 5.8: ML-KEM-512 energy per operation for energy-measured embedded platforms.

### 5.3 Memory Usage Results

This section compares flash and RAM usage across the MCU platforms. Stack usage was not included in the final analysis because a consistent cross-platform measurement method was not available for every target.

As seen in Table 5.1, the ESP32-C61 has higher memory usage than the other boards, sometimes by an order of magnitude, likely due to the platforms larger software and runtime footprint. On the STM32, Pico, and nRF52840 DK, ML-KEM-512 falls between the smaller AEAD implementations, such as AES-128-GCM and GIFT-COFB, and the larger Ascon variants in terms of flash usage. Across those same three platforms, the reported RAM values are largely consistent regarding the AEAD implementations, while ML-KEM-512 differs more substantially. This suggests that, in the final binary reported by ORBIT, platform-level runtime and toolchain effects contribute alongside algorithm-specific code size. Regarding



Table 5.1: Flash and RAM usage across MCU platforms and algorithms.

| Platform     | Algorithm     | Flash (bytes) | RAM (bytes) |
|--------------|---------------|---------------|-------------|
| ESP32-C61    | Ascon-AEAD128 | 218458        | 426102      |
| ESP32-C61    | Ascon-80pq    | 214162        | 426094      |
| ESP32-C61    | GIFT-COFB     | 197042        | 426086      |
| ESP32-C61    | AES-128-GCM   | 194132        | 426646      |
| ESP32-C61    | ML-KEM-512    | 199036        | 380430      |
| STM32 F446RE | Ascon-AEAD128 | 40628         | 68616       |
| STM32 F446RE | Ascon-80pq    | 37812         | 68616       |
| STM32 F446RE | GIFT-COFB     | 25768         | 68616       |
| STM32 F446RE | AES-128-GCM   | 23756         | 68616       |
| STM32 F446RE | ML-KEM-512    | 27712         | 22640       |
| RP2040 Pico  | Ascon-AEAD128 | 63048         | 54300       |
| RP2040 Pico  | Ascon-80pq    | 59864         | 54300       |
| RP2040 Pico  | GIFT-COFB     | 45704         | 54300       |
| RP2040 Pico  | AES-128-GCM   | 43712         | 54616       |
| RP2040 Pico  | ML-KEM-512    | 50232         | 8692        |
| nRF52840 DK  | Ascon-AEAD128 | 37368         | 51648       |
| nRF52840 DK  | Ascon-80pq    | 34504         | 51648       |
| nRF52840 DK  | GIFT-COFB     | 22472         | 51648       |
| nRF52840 DK  | AES-128-GCM   | 20408         | 51648       |
| nRF52840 DK  | ML-KEM-512    | 24732         | 5668        |

the smaller MCU targets, the nRF52840 DK generally yields the smallest flash footprint, while the Pico and the STM32 occupy an intermediate range depending on the algorithm being run.

## 5.4 Timing Consistency Results

This section summarizes the consistency of AEAD and ML-KEM benchmarks across runs. Timing consistency is reported using the coefficient of variation (CV), computed from the standard deviation and mean of per-run latency values.

Table 5.2: Representative timing consistency results across platforms.

| Platform       | Measurement                | Mean ( $\mu$ s) | SD ( $\mu$ s) | CV (%) |
|----------------|----------------------------|-----------------|---------------|--------|
| ESP32-C61      | Ascon-AEAD128 enc @ 1024 B | 670.627         | 0.005926      | 0.0009 |
| ESP32-C61      | ML-KEM-512 KeyGen          | 5887.040        | 0.067949      | 0.0012 |
| ESP32-C61      | ML-KEM-512 Encap           | 6570.859        | 0.120906      | 0.0018 |
| ESP32-C61      | ML-KEM-512 Decap           | 7760.851        | 0.278199      | 0.0036 |
| STM32 F446RE   | Ascon-AEAD128 enc @ 1024 B | 639.255         | 0.000000      | 0.0000 |
| STM32 F446RE   | ML-KEM-512 KeyGen          | 5653.987        | 0.000000      | 0.0000 |
| STM32 F446RE   | ML-KEM-512 Encap           | 6243.373        | 0.000000      | 0.0000 |
| STM32 F446RE   | ML-KEM-512 Decap           | 7418.374        | 0.000000      | 0.0000 |
| RP2040 Pico    | Ascon-AEAD128 enc @ 1024 B | 1467.929        | 0.081930      | 0.0056 |
| RP2040 Pico    | ML-KEM-512 KeyGen          | 10209.855       | 0.755264      | 0.0074 |
| RP2040 Pico    | ML-KEM-512 Encap           | 11925.618       | 0.751886      | 0.0063 |
| RP2040 Pico    | ML-KEM-512 Decap           | 14447.218       | 1.590455      | 0.0110 |
| nRF52840 DK    | Ascon-AEAD128 enc @ 1024 B | 2058.455        | 0.000000      | 0.0000 |
| nRF52840 DK    | ML-KEM-512 KeyGen          | 17237.144       | 0.000000      | 0.0000 |
| nRF52840 DK    | ML-KEM-512 Encap           | 19842.481       | 0.000000      | 0.0000 |
| nRF52840 DK    | ML-KEM-512 Decap           | 24182.781       | 0.000000      | 0.0000 |
| Raspberry Pi 5 | Ascon-AEAD128 enc @ 1024 B | 5.792           | 0.027731      | 0.4788 |
| Raspberry Pi 5 | ML-KEM-512 KeyGen          | 47.209          | 0.267921      | 0.5675 |
| Raspberry Pi 5 | ML-KEM-512 Encap           | 58.704          | 0.348290      | 0.5933 |
| Raspberry Pi 5 | ML-KEM-512 Decap           | 77.757          | 0.399647      | 0.5140 |

As shown in Table 5.2, the variation between runs is very low across all platforms. The MCU

targets generally have a small CV, well below 0.01%, indicating highly stable, repeatable measurements under the ORBIT benchmark harness. The Raspberry Pi 5 is again an outlier here, with a particularly high CV, though still below 1% for the AEAD and ML-KEM measurements presented here. For the STM32 and the nRF52840 DK, multiple entries show no observed standard deviation at the recorded precision, indicating that there was no measurable variance between runs in these cases.

## 5.5 Post-Quantum Overhead

### 5.5.1 Ascon-80pq Overhead

This subsection compares the post-quantum Ascon-80pq variant with the earlier Ascon-AEAD128 baseline in order to estimate the incremental adoption of the higher-security variant.

When comparing Figure 5.9 to Figure 5.6, we can see that the energy overhead is very platform dependent. On the ESP32-C61 and Pico, Ascon-80pq energy per byte at 1024 bytes increases by roughly 26% and 33% respectively when compared to Ascon-AEAD128. The Nordic board is a notable exception here, as Ascon-80pq actually measures lower energy per byte than Ascon-AEAD128 over most of the tested range.

Looking back to Figure 5.4, the cycles per byte overhead is much more consistent across the board when compared to the energy overhead. For the ESP32-C61, it increases by 30.73% from 104.785421 to 136.983529; the STM32 increases by 30.45% from 112.36090 to 146.586633; the Pico increases by 31.37% from 179.190586 to 235.408249; and the nRF52840 increases by the lowest amount, 26.98% from 128.653430 to 163.362414.

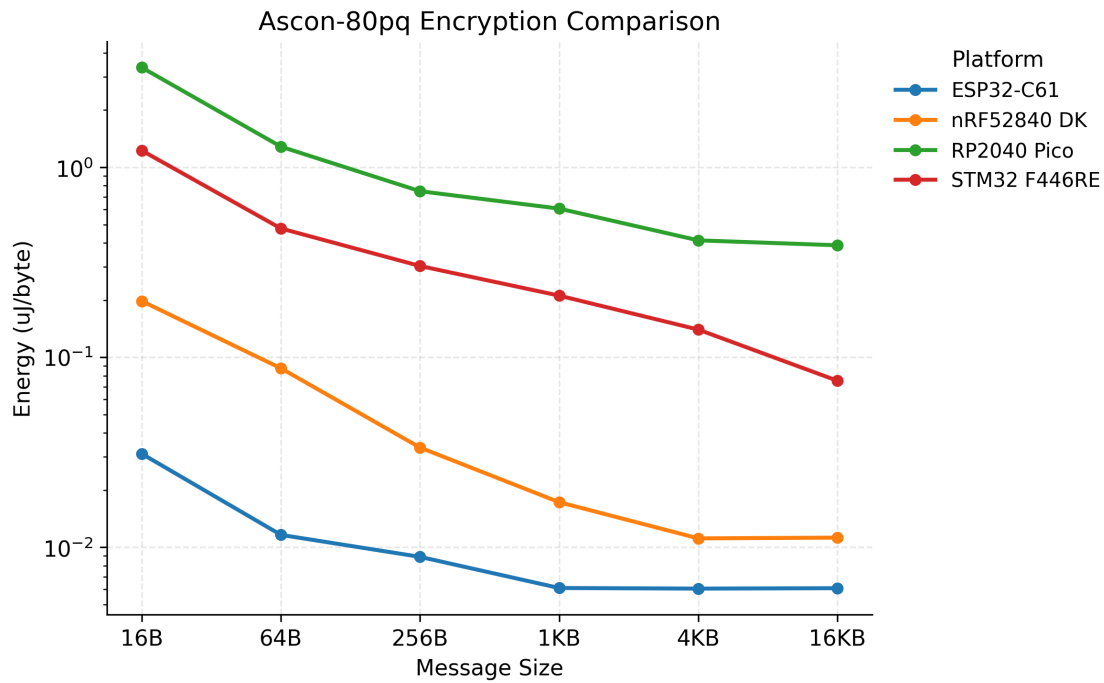


Figure 5.9: Ascon-80pq encryption energy per byte across message sizes for energy-measured embedded platforms.

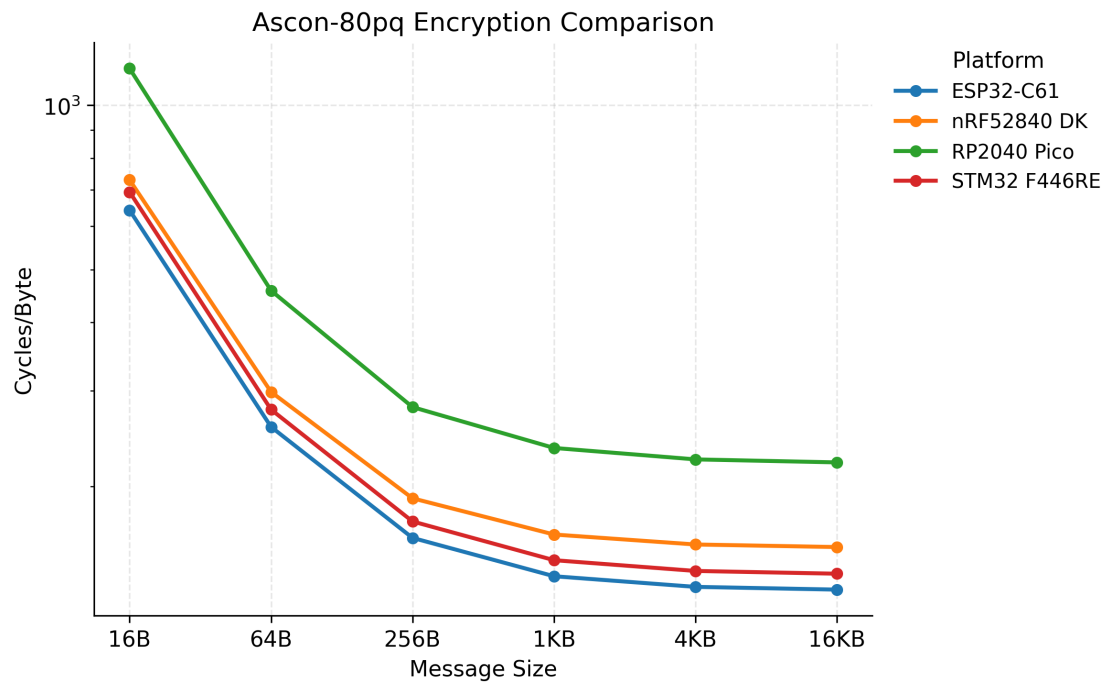


Figure 5.10: Ascon-80pq encryption cycles per byte across message sizes for MCU-class platforms.

### 5.5.2 ML-KEM-512 Overhead

Unlike Ascon-80pq, ML-KEM-512 is a functionally different algorithm rather than a variant of an existing AEAD primitive. Because of this, its overhead is better interpreted as the additional cost of introducing post-quantum public-key operations into the system.

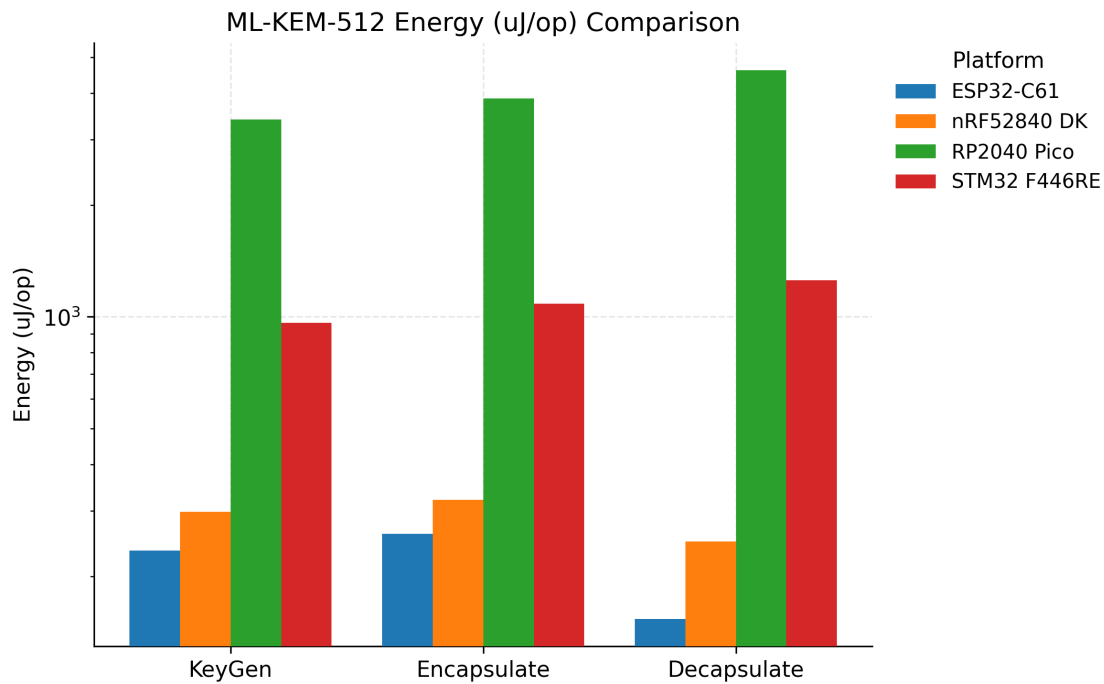


Figure 5.11: ML-KEM-512 energy per operation for energy-measured embedded platforms.

Looking at Figure 5.11, the per-operation energy of ML-KEM is much higher than the per-message energy cost of the lightweight AEADs. With the ESP32-C61, the operations range from 153.60 to 259.86  $\mu\text{J}/\text{op}$ , compared to the Pico, which ranges from 3399.99 to 4609.41  $\mu\text{J}/\text{op}$ . This means that the Pico has a Decapsulation cost roughly 30 times higher than on the ESP32-C61. The STM32 and nRF52840 DK fall between the two extremes, with the ESP32-C61 and nRF52840 DK forming the lower-energy ML-KEM group. This highlights that the cost of adding post-quantum public-key operations is far more substantial than the incremental overhead observed between the symmetric AEAD variants.

# Chapter 6

## Discussion

The results in Chapter 5 show that no single platform is best across all metrics. Instead, the study shows tradeoffs between performance, energy efficiency, memory usage, and post-quantum overhead. Lightweight AEAD algorithms remain well-suited to frequent message protection on constrained devices, whereas ML-KEM-512 incurs substantially higher costs across the board and would be much more appropriate for infrequent public-key operations, such as key establishment. The results also show that the choice of platform plays a major role in how practical these algorithms will truly be in real-life deployment. This chapter discusses those tradeoffs, considers their implications for real-world IoT systems, and outlines the limitations that shape how these findings should be interpreted.

### 6.1 Tradeoff Analysis

The measured trade-offs become clearest when comparing software AES-128-GCM with the lightweight AEAD algorithms and when separating symmetric post-quantum overhead from the much larger post-quantum public-key operations. Across the MCU targets, both algorithm and platform choices strongly affect the practicality of a cryptographic design.

Software AES-GCM proved to be dramatically more expensive than lightweight AEAD. This remained true throughout cycles/byte, latency, and energy per byte, making AEAD the practical default for constrained endpoints. When it came to cycles on the STM32,

Ascon-AEAD128 came in at 112.4 cycles per byte, whereas AES-GCM reached a staggering 26385 cycles per byte. On the energy front, the ESP32-C61 ran Ascon-AEAD128 at 0.00486  $\mu\text{J}/\text{byte}$  and AES-GCM at 0.8144  $\mu\text{J}/\text{byte}$ , indicating a significant difference between these implementations.

Both Ascon-AEAD128 and GIFT-COFB are good candidates, and it makes sense that they both made it to the finals in the standardization of lightweight cryptography. Neither of these dominates the other across the board, and they remain quite comparable throughout testing. Though Ascon-AEAD128 will remain the stronger general default because of its balance and its status as the NIST LWC standard.

When comparing Ascon-AEAD128 to its quantum-hardened counterpart, Ascon-80pq, the additional cost is noticeable, though still much smaller than the cost of moving to a full post-quantum setup such as ML-KEM-512. In cycles per byte, the overhead is fairly consistent across all MCU targets at 1024 bytes, increasing by 30.73% on the ESP32-C61, 30.45% on the STM32, 31.37% on the Pico, and 26.98% on the nRF52840 when compared to Ascon-AEAD128. The overhead when looking at the energy is much less uniform, with the ESP32-C61 and Pico showing higher costs with Ascon-80pq, while the Nordic board measured a lower energy per byte in the same scenario. Overall, Ascon-80pq appears to be a good post-quantum symmetric hedge, though it still introduces a noticeable penalty when compared with the baseline Ascon-AEAD128 design.

Beyond the algorithms themselves, platform choice plays a major role in determining how practical a cryptographic design can be. The ESP32-C61 has emerged as the strongest all-around platform in the dataset, with excellent AEAD timing and the lowest measured energy costs across the tested MCU platforms. The STM32 proved to be quite strong in latency, with Ascon-AEAD128 completing in 639.3  $\mu\text{s}$  for 1024 bytes compared with 670.6  $\mu\text{s}$  on the ESP32-C61, but it is less attractive in energy-sensitive scenarios. The nRF52840 DK

is not as strong in raw timing, though it performs well in energy efficiency, making it a compelling option for battery life or energy budget, rather than for absolute throughput. The Pico, while workable across all measured algorithms, is generally less competitive than the other platforms benchmarked here in both performance and energy. The Raspberry Pi 5, meanwhile, belongs to a wholly different class entirely and is best interpreted as a gateway reference rather than a direct MCU peer.

ML-KEM-512 represents a qualitatively different cost category when compared to the symmetric algorithms. While Ascon-80pq brings a moderate overhead relative to Ascon-AEAD128, ML-KEM-512 imposes a much larger latency and energy burden across all MCU targets. On the ESP32-C61, ML-KEM-512 operations range from 153.60 to 259.86  $\mu\text{J}/\text{op}$ , while the nRF52840 ranges from 247.96 to 321.44  $\mu\text{J}/\text{op}$ . These energy costs increase drastically when looking at the STM32, which ranges from 962.42 to 1255.05  $\mu\text{J}/\text{op}$ , and even more so when it comes to the Pico, where they range from 3399.99 to 4609.41  $\mu\text{J}/\text{op}$ . This indicates that ML-KEM-512 is feasible on constrained hardware, though it should be treated as a tool for infrequent key establishment rather than for routine message protection. This makes the cost of post-quantum public-key adoption substantially higher than that of adopting post-quantum symmetric primitives such as Ascon-80pq, though with this added cost comes much deeper security.

To provide a security-normalized view of efficiency, additional derived metrics were computed from the summary datasets and are reported in Appendices B and C. These metrics normalize measured cycle counts and energy costs by the claimed or standardized security strength of each primitive in testing, yielding security bits per cycle and security bits per microjoule. Appendix B reports a classical or standardized normalization, using a 128-bit basis for AES-128-GCM, Ascon-AEAD128, GIFT-COFB, and Ascon-80pq, while ML-KEM-512 is normalized using NIST Category 1 as an AES-128-equivalent comparison target. Ap-



pendix C reports an alternate quantum-search-adjusted normalization for the symmetric schemes, using 64 bits for the 128-bit symmetric designs and 80 bits for Ascon-80pq. These derived metrics do not directly measure cryptographic security, though they provide a useful comparative lens for evaluating how each platform translates computational and energy costs into standardized security targets.

## 6.2 Practical Deployment Implications

Different deployment profiles need to prioritize different constraints. These include energy, latency, memory, and long-term security requirements. Because of this, no single algorithm-platform combination is the best for each and every IoT role. This section will walk through representative deployment profiles one by one.

Lightweight AEAD and Post-Quantum public-key operations impose vastly different burdens; thus, practical recommendations should distinguish between routine message protection and infrequent key establishment.

When it comes to wearable devices, battery life is the dominant constraint. Low memory usage and latency are desirable, though they are not as important as energy efficiency. In this setting, frequent public-key operations are harder to justify, since they incur a much higher cost than routine symmetric protection. Because of this, lightweight AEAD is strongly preferred over using software AES-GCM. Ascon-AEAD128 is the clean default recommendation here. The ESP32-C61 and nRF52840 DK are the best fits due to their strong energy-efficient nature, with Ascon-AEAD128 measured at 0.00486  $\mu\text{J}/\text{byte}$  and 0.02895  $\mu\text{J}/\text{byte}$ , respectively, at 1024 bytes. The use of ML-KEM-512, if at all, should be limited to infrequent setup or re-keying operations.

For smart home endpoints, the constraints are much more balanced across latency, energy, and implementation practicality. While these devices still benefit greatly from lightweight AEAD, they can tolerate more computational cost than wearable-class devices. Ascon-AEAD128 and GIFT-COFB would both be reasonable choices here, while software AES-GCM remains quite a bit more expensive. In this scenario, the ESP32-C61 and STM32 are quite attractive when latency is a priority, with Ascon-AEAD128 latencies of 670.6  $\mu$ s and 639.3  $\mu$ s, respectively, at 1024 bytes. The nRF52840 DK also remains a good candidate in scenarios where energy matters more than speed alone. ML-KEM-512 would be feasible here for something like session establishment, but again, should not be treated as a routine operation.

For industrial nodes, long-term deployment and security planning are much more important than for short-lived consumer devices, which often get upgraded within a year or even just a few months. Ascon-80pq would therefore prove to be the more practical post-quantum symmetric hedge, with a cycles-per-byte overhead of 30.73% at 1024 bytes on the ESP32-C61, 30.45% on the STM32, and 26.98% on the nRF52840 DK relative to Ascon-AEAD128. In this scenario, ML-KEM-512 would also be somewhat viable, though it should be used for infrequent key establishment operations. The ESP32-C61 and STM32 appear to be the strongest fits here, particularly where both performance and long-term migration planning are important.

For gateway or border-node deployments, raw performance and flexibility matter much more than tight energy constraints. In this role, the Raspberry Pi 5 would be the clear choice, as it handles both lightweight and post-quantum workloads much better than the MCU targets. For example, it completes Ascon-AEAD128 encryption in 5.79  $\mu$ s at 1024 bytes, while ML-KEM-512 KeyGen and Decapsulation take 47.2  $\mu$ s and 77.8  $\mu$ s respectively. Even heavier cryptographic operations are much easier to accommodate in this context, making

this class the most natural place for frequent post-quantum public-key operations.

For constrained endpoints that still need some degree of post-quantum readiness, the main issue is balancing future-facing security goals against the strict MCU resource limits. Ascon-80pq would be a good fit in this case, considering its overhead is much smaller when compared to ML-KEM-512, although Ascon-80pq should be understood as a quantum hedge rather than a full replacement for post-quantum public-key mechanisms such as ML-KEM-512. ML-KEM-512 remains possible, though it is costly enough that it should again be reserved for infrequent operations. The ESP32-C61 and nRF52840 would be the best fits for this profile, with ML-KEM operation costs ranging from 153.60 to 259.86  $\mu\text{J}/\text{op}$  on the ESP32 and from 247.96 to 321.44  $\mu\text{J}/\text{op}$  on the nRF52840, far below any values measured on the Pico.

Table 6.1 summarizes these deployment-oriented recommendations.

Table 6.1: Compact deployment-oriented suitability matrix derived from measured timing, energy, memory, and post-quantum overhead results.

| Profile             | Preferred AEAD               | ML-KEM Viability | Best-Fit Platforms            |
|---------------------|------------------------------|------------------|-------------------------------|
| Wearable endpoint   | Ascon-AEAD128                | Limited          | ESP32-C61, nRF52840 DK        |
| Smart home endpoint | Ascon-AEAD128<br>GIFT-COFB   | / Feasible       | ESP32-C61, STM32, nRF52840 DK |
| Industrial node     | Ascon-AEAD128<br>Ascon-80pq  | / Selective      | ESP32-C61, STM32              |
| Gateway node        | Ascon-AEAD128<br>AES-128-GCM | / Strong         | Raspberry Pi 5                |
| PQ-aware endpoint   | Ascon-80pq                   | Possible         | ESP32-C61, nRF52840 DK        |

## 6.3 Overall Limitations

This study was software-only, and the use of hardware acceleration was not a focus. This choice was made to preserve comparability across platforms, meaning that the reported results reflect portable software implementations, rather than the best possible vendor-optimized deployments.

Energy measurements were conducted at the board level rather than as an isolated core-only silicon power draw, so rail and instrumentation differences across boards may affect the reported absolute values. This does not reduce the usefulness of the results, but it does mean they should be interpreted as platform-level energy costs.

The Raspberry Pi 5 was intentionally included as a distinct class of platform from the other MCU targets. Though this does lead to differences due to Linux scheduling and runtime effects. While this makes it a great gateway reference, it is not directly comparable to its bare-metal peers.

Stack usage was excluded from the results because a consistent cross-platform method was not available. The memory analysis, therefore, focuses solely on flash and RAM use.

The results here depend on the chosen implementations, compiler flags, and the SDK/runtime/toolchain behavior. Using different optimized implementations could therefore lead to shifts in relative rankings across the results.

In addition, iteration counts were adjusted by platform and workload rather than being fixed uniformly, to keep heavier cryptographic operations practical to benchmark while still preserving stable aggregate statistics.

Finally, security-normalized efficiency metrics are derived rather than directly measured. They are based on claimed or standardized security targets and are intended as an aid for

comparison, rather than a direct measurement of cryptographic strength under real-world attack models.

# Chapter 7

## Conclusions & Future Work

### 7.1 Conclusions

The primary objective of this research project is to analyze the computational, energy-efficiency, and memory-utilization characteristics of lightweight and post-quantum cryptography techniques on constrained IoT devices. It is apparent from the presented results that all of these algorithms are viable options. Still, depending on the particular implementation and use case, they come at different costs..

Ascon-AEAD128, the NIST LWC approved standard, is an outstanding algorithm in both computational performance and energy efficiency across all MCU targets. The second finalist of the contest, GIFT-COFB, remained close to Ascon-AEAD128 across most parameters and even surpassed it in some scenarios. Compared with AES-128-GCM, Ascon-AEAD128 and GIFT-COFB have shown higher performance and energy efficiency in throughput, latency, cycles per byte, and memory usage. Ascon-80pq, an investigated symmetric post-quantum hedge, has demonstrated additional but acceptable overhead in comparison to Ascon-AEAD128. The ML-KEM-512 implementation has proven too costly, demanding significantly more time and energy, and is viable only for periodic public-key encryption, rather than constant message security.

Both the algorithm and platform selection have made a critical impact on the actual feasibil-

ity of these cryptographic tools. The ESP32-C61 and nRF52840 DK are the best performers in terms of energy efficiency. The STM32 platform was superior in timing and latency metrics. Security efficiency metrics in Appendices B and C confirm these findings.

In addition to the presented analysis of selected symmetric and post-quantum techniques, this work provides the ORBIT benchmarking framework, a multi-platform empirical dataset spanning timing, energy, memory, and consistency measures, a deployment-focused suitability matrix, and derived security efficiency metrics for comparative analysis. The results of this research show that AEAD should be used as a default symmetric protection layer on constrained IoT deployments for the near future. Ascon-80pq appears to be an appropriate symmetric post-quantum hedge in certain scenarios, while ML-KEM-512 may be used rarely due to higher overheads.

## 7.2 Future Work

Several natural extensions follow from this work:

- Evaluate the same algorithm set with hardware acceleration enabled where available, in order to compare portable software results against vendor-optimized deployments.
- Extend the post-quantum analysis to additional KEM and signature schemes, allowing a broader comparison of public-key post-quantum options on constrained hardware.
- Integrate the evaluated primitives into higher-level protocols or application stacks, such as TLS, DTLS, or IoT-specific secure messaging frameworks, to measure system-level overhead beyond what is gathered here.
- Expand the security-normalized analysis by exploring additional normalization schemes

and security models, particularly for post-quantum comparisons across symmetric and public-key primitives.

- Investigate the impact of alternative compilers, optimization flags, SDK versions, and hand-optimized implementations on the relative rankings observed in this study.
- Apply ORBIT to a broader range of platforms and deployment scenarios, including newer MCU families, heterogeneous edge nodes, and energy-constrained real-world workloads.



# Bibliography

- [1] S. Sinha, “Number of connected IoT devices growing 14% to 21.1 billion,” IoT Analytics, Oct. 2025, [Online]. Available: <https://iot-analytics.com/number-connected-iot-devices/>. [Accessed: Mar. 2026].
- [2] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Partridge, P. Pearce, N. Sullivan, K. Thomas, and Y. Zhou, “Understanding the Mirai botnet,” in *Proceedings of the 26th USENIX Security Symposium*. USENIX Association, 2017, pp. 1093–1110, [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>. [Accessed: Mar. 2026].
- [3] O. Yoachimik and J. Pacheco, “2025 Q4 DDoS threat report: A record-setting 31.4 Tbps attack caps a year of massive DDoS assaults,” Cloudflare Blog, Feb. 2026, [Online]. Available: <https://blog.cloudflare.com/ddos-threat-report-2025-q4/>. [Accessed: Apr. 2026].
- [4] M. Fagan, K. Megas, K. Scarfone, and M. Smith, “Foundational cybersecurity activities for IoT device manufacturers,” National Institute of Standards and Technology, Tech. Rep. NISTIR 8259, May 2020, [Online]. Available: <https://doi.org/10.6028/NIST.IR.8259>. [Accessed: Mar. 2026].
- [5] —, “IoT device cybersecurity capability core baseline,” National Institute of Standards and Technology, Tech. Rep. NISTIR 8259A, May 2020, [Online]. Available: <https://doi.org/10.6028/NIST.IR.8259A>. [Accessed: Mar. 2026].

- [6] National Institute of Standards and Technology, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” National Institute of Standards and Technology, Tech. Rep. SP 800-38D, 2007, [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-38D>. [Accessed: Mar. 2026].
- [7] P. Schwabe and K. Stoffelen, “All the AES you need on Cortex-M3 and M4,” in *Selected Areas in Cryptography — SAC 2016*, ser. Lecture Notes in Computer Science, vol. 10532. Springer, 2016, pp. 180–194, [Online]. Available: <https://eprint.iacr.org/2016/714.pdf>. [Accessed: Mar. 2026].
- [8] D. J. Bernstein, “Cache-timing attacks on AES,” 2005, [Online]. Available: <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>. [Accessed: Mar. 2026].
- [9] National Institute of Standards and Technology, “Lightweight cryptography standardization process: NIST selects Ascon,” National Institute of Standards and Technology, Tech. Rep., 2023, [Online]. Available: <https://www.nist.gov/news-events/news/2023/02/lightweight-cryptography-standardization-process-nist-selects-ascon>. [Accessed: Mar. 2026].
- [10] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, “Ascon v1.2: Lightweight authenticated encryption and hashing,” Submission to NIST Lightweight Cryptography Standardization, 2021, [Online]. Available: <https://ascon.iaik.tugraz.at/>. [Accessed: Mar. 2026].
- [11] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134.
- [12] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, “Transition to post-quantum cryptography standards,” National Institute of Standards and Technology,

- Tech. Rep. NIST IR 8547, Nov. 2024, initial Public Draft. [Online]. Available: <https://doi.org/10.6028/NIST.IR.8547.ipd>. [Accessed: Mar. 2026].
- [13] R. Chen *et al.*, “Report on post-quantum cryptography,” National Institute of Standards and Technology, Tech. Rep. NISTIR 8105, 2016, [Online]. Available: <https://doi.org/10.6028/NIST.IR.8105>. [Accessed: Mar. 2026].
- [14] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” National Institute of Standards and Technology, Tech. Rep. FIPS 203, 2024, [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.203>. [Accessed: Mar. 2026].
- [15] —, “Module-lattice-based digital signature standard,” National Institute of Standards and Technology, Tech. Rep. FIPS 204, 2024, [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.204>. [Accessed: Mar. 2026].
- [16] —, “Stateless hash-based digital signature standard,” National Institute of Standards and Technology, Tech. Rep. FIPS 205, 2024, [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.205>. [Accessed: Mar. 2026].
- [17] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4,” in *NIST Second PQC Standardization Conference*, 2019, [Online]. Available: <https://eprint.iacr.org/2019/844>. [Accessed: Mar. 2026].
- [18] C. Bormann, M. Ersue, and A. Keranen, “Terminology for constrained-node networks,” Internet Engineering Task Force, RFC 7228, 2014, [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7228>. [Accessed: Mar. 2026].
- [19] R. Rusch, “Companies are flying old satellites longer, study

- finds,” *SpaceNews*, 2021, [Online]. Available: <https://spacenews.com/companies-are-flying-old-satellites-longer-study-finds/>. [Accessed: Mar. 2026].
- [20] Bright Ascension, “Everything you need to know about CubeSats,” 2023, [Online]. Available: <https://brightascension.com/everything-you-need-to-know-about-cubesats-2/>. [Accessed: Mar. 2026].
- [21] National Institute of Standards and Technology, “Lightweight cryptography standardization process,” National Institute of Standards and Technology, Tech. Rep., 2025, [Online]. Available: <https://csrc.nist.gov/projects/lightweight-cryptography>. [Accessed: Mar. 2026].
- [22] M. Sönmez Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, “Ascon-based lightweight cryptography standards for constrained devices,” National Institute of Standards and Technology, NIST Special Publication 800-232, 2025, [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-232>. [Accessed: Mar. 2026].
- [23] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1996, pp. 212–219.
- [24] S. Banik, A. Chakraborti, A. Inoue, T. Iwata, K. Minematsu, M. Nandi, T. Peyrin, Y. Sasaki, S. M. Sim, and Y. Todo, “GIFT-COFB v1.0,” in *Submission to the NIST Lightweight Cryptography Standardization Process*, 2019, [Online]. Available: <https://csrc.nist.gov/Projects/lightweight-cryptography/finalists>. [Accessed: Mar. 2026].
- [25] S. Renner, E. Pozzobon, and J. Mottok, “The final round: Benchmarking NIST LWC ciphers on microcontrollers,” in *Attacks and Defenses for the Internet-of-Things (ADIoT 2022)*, ser. Lecture Notes in Computer Science, vol. 13745. Springer, 2022, [Online]. Available: [https://doi.org/10.1007/978-3-031-21311-3\\_1](https://doi.org/10.1007/978-3-031-21311-3_1). [Accessed: Mar. 2026].

- [26] National Security Agency, “Commercial national security algorithm suite 2.0,” National Security Agency, Tech. Rep., 2022, [Online]. Available: [https://media.defense.gov/2022/Sep/07/2003071836/-1/-1/0/CSA\\_CNSA\\_2.0\\_ALGORITHMS\\_.PDF](https://media.defense.gov/2022/Sep/07/2003071836/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF). [Accessed: Mar. 2026].
- [27] L. Botros, M. J. Kannwischer, and P. Schwabe, “Memory-efficient high-speed implementation of Kyber on Cortex-M4,” in *Progress in Cryptology — LATINCRYPT 2019*, ser. Lecture Notes in Computer Science. Springer, 2019, [Online]. Available: [https://doi.org/10.1007/978-3-030-30530-7\\_11](https://doi.org/10.1007/978-3-030-30530-7_11). [Accessed: Mar. 2026].
- [28] D. Atkins, “Requirements for post-quantum cryptography on embedded devices in the IoT,” in *Third NIST Post-Quantum Cryptography Standardization Conference*, 2021, [Online]. Available: <https://csrc.nist.gov/CSRC/media/Events/third-pqc-standardization-conference/documents/accepted-papers/atkins-requirements-pqc-iot-pqc2021.pdf>. [Accessed: Mar. 2026].
- [29] National Institute of Standards and Technology, “NIST selects HQC as fifth algorithm for post-quantum encryption,” National Institute of Standards and Technology, Tech. Rep., Mar. 2025, [Online]. Available: <https://www.nist.gov/news-events/news/2025/03/nist-selects-hqc-fifth-algorithm-post-quantum-encryption>. [Accessed: Mar. 2026].
- [30] D.-D. Dinu, A. Biryukov, J. Großschädl, D. Khovratovich, Y. Le Corre, and L. Perrin, “FELICS — fair evaluation of lightweight cryptographic systems,” in *NIST Workshop on Lightweight Cryptography*, 2015, [Online]. Available: <https://csrc.nist.gov/csrc/media/events/lightweight-cryptography-workshop-2015/documents/papers/session7-dinu-paper.pdf>. [Accessed: Mar. 2026].
- [31] Raspberry Pi Ltd, *RP2040 Datasheet*, <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024, accessed: 2026.

- [32] ARM Limited, *Cortex-M0+ Technical Reference Manual*, <https://developer.arm.com/documentation/ddi0484/latest>, 2012, accessed: 2026.
- [33] Nordic Semiconductor. (2026) nRF52840 System on Chip. Accessed: 2026. [Online]. Available: <https://www.nordicsemi.com/Products/nRF52840>
- [34] ARM Limited, *Cortex-M4 Technical Reference Manual*, <https://developer.arm.com/documentation/ddi0439/latest>, 2010, accessed: 2026.
- [35] STMicroelectronics, *STM32F446xC/E Datasheet*, <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>, 2024, accessed: 2026.
- [36] Espressif Systems, *ESP32-C61 esp-dev-kits Documentation*, <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c61/esp-dev-kits-en-master-esp32c61.pdf>, 2026, accessed: 2026.
- [37] Raspberry Pi Ltd, *Raspberry Pi 5 Product Brief*, <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>, 2023, accessed: 2026.
- [38] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, <https://github.com/riscv/riscv-isa-manual/releases/latest>, 2024, accessed: 2026.

# Appendices

# Appendix A

## Target Toolchain and Build Environment



Table A.1: Target-specific toolchains and flashing dependencies.

| Target            |          | Toolchain or dependency                    |        |     | Purpose                                                                                         |
|-------------------|----------|--------------------------------------------|--------|-----|-------------------------------------------------------------------------------------------------|
| Raspberry Pi Pico |          | Pico SDK                                   |        |     | Provides RP2040 startup code, board support, and CMake integration for Pico firmware builds.    |
| Raspberry Pi Pico |          | arm-none-eabi-gcc                          |        |     | Cross-compiles ORBIT firmware for the Cortex-M0+ target.                                        |
| Raspberry Pi Pico |          | picotool                                   |        |     | Reboots the Pico into BOOTSEL mode and supports automated UF2 flashing workflows.               |
| Raspberry Pi Pico |          | /mnt/pico mount point and attach\_pico.ps1 |        |     | Supports WSL2 USB attach and UF2 copy-based flashing.                                           |
| STM32 F446RE      | Nucleo   | STM32CubeF4                                |        |     | Provides STM32F4 device headers, startup files, and HAL/CMSIS support used by the STM32 target. |
| STM32 F446RE      | Nucleo   | arm-none-eabi-gcc                          |        |     | Cross-compiles ORBIT firmware for the Cortex-M4 target.                                         |
| STM32 F446RE      | Nucleo   | OpenOCD                                    |        |     | Flashes and resets the board through the ST-LINK interface.                                     |
| Nordic PCA10056   | nRF52840 | nRF5 SDK                                   |        |     | Provides Nordic device support files and board definitions for the nRF52 target.                |
| Nordic PCA10056   | nRF52840 | arm-none-eabi-gcc                          |        |     | Cross-compiles ORBIT firmware for the Cortex-M4F target.                                        |
| Nordic PCA10056   | nRF52840 | Nordic Command Line Tools / nrfjprog       |        |     | Erases, programs, verifies, and resets the nRF52840 board.                                      |
| Nordic PCA10056   | nRF52840 | Segger J-Link tools                        |        |     | Provides host support for the PCA10056 debug and virtual COM interface.                         |
| ESP32-C61         |          | ESP-IDF v5.5.3                             |        |     | Provides the Espressif build system, target support, flashing, and monitor workflow.            |
| ESP32-C61         |          | idf.py                                     |        |     | Configures, builds, flashes, and monitors the ESP-IDF target.                                   |
| ESP32-C61         |          | ESP-IDF toolchain                          | RISC-V | GCC | Cross-compiles ORBIT for the RV32IMAC target.                                                   |
| ESP32-C61         |          | esptool.py                                 |        |     | Supports ESP32 flashing as part of the ESP-IDF workflow.                                        |
| Raspberry Pi 5    |          | Native Linux GCC/build-essential toolchain |        |     | Builds ORBIT as a native Linux executable on the Raspberry Pi 5.                                |
| Raspberry Pi 5    |          | Python and CMake                           |        |     | Runs orbit.py, configures the local build, executes the benchmark, and captures stdout.         |

# Appendix B

## Classical / Standardized Normalization

### B.1 AEAD at 1024 B

Table B.1: Security-normalized AEAD efficiency at 1024 bytes under classical or standardized security assumptions.

| Platform     | Algorithm     | Sec. Bits | Bits/Cycle | Bits/ $\mu$ J |
|--------------|---------------|-----------|------------|---------------|
| ESP32-C61    | AES-128-GCM   | 128       | 7.67e-7    | 0.1535        |
| ESP32-C61    | Ascon-AEAD128 | 128       | 2.39e-6    | 25.7199       |
| ESP32-C61    | Ascon-80pq    | 128       | 1.83e-6    | 20.4798       |
| ESP32-C61    | GIFT-COFB     | 128       | 2.16e-6    | 22.2644       |
| STM32 F446RE | AES-128-GCM   | 128       | 9.48e-7    | 0.0033        |
| STM32 F446RE | Ascon-AEAD128 | 128       | 2.22e-6    | 0.6501        |
| STM32 F446RE | Ascon-80pq    | 128       | 1.71e-6    | 0.5922        |
| STM32 F446RE | GIFT-COFB     | 128       | 2.28e-6    | 0.6443        |
| RP2040 Pico  | AES-128-GCM   | 128       | 6.62e-7    | 0.0016        |
| RP2040 Pico  | Ascon-AEAD128 | 128       | 1.40e-6    | 0.2734        |
| RP2040 Pico  | Ascon-80pq    | 128       | 1.06e-6    | 0.2063        |

| Platform       | Algorithm     | Sec. Bits | Bits/Cycle | Bits/ $\mu$ J |
|----------------|---------------|-----------|------------|---------------|
| RP2040 Pico    | GIFT-COFB     | 128       | 1.38e-6    | 0.3088        |
| nRF52840 DK    | AES-128-GCM   | 128       | 6.53e-7    | 0.0134        |
| nRF52840 DK    | Ascon-AEAD128 | 128       | 1.94e-6    | 4.3174        |
| nRF52840 DK    | Ascon-80pq    | 128       | 1.53e-6    | 7.2393        |
| nRF52840 DK    | GIFT-COFB     | 128       | 2.07e-6    | 13.3617       |
| Raspberry Pi 5 | AES-128-GCM   | 128       | 1.39e-4    | –             |
| Raspberry Pi 5 | Ascon-AEAD128 | 128       | 8.18e-4    | –             |
| Raspberry Pi 5 | Ascon-80pq    | 128       | 6.72e-4    | –             |
| Raspberry Pi 5 | GIFT-COFB     | 128       | 2.48e-4    | –             |

No energy measurements were collected for the Raspberry Pi 5, so Bits/ $\mu$ J values are not reported for that platform.

## B.2 ML-KEM-512

Table B.2: Security-normalized ML-KEM-512 efficiency under classical or standardized security assumptions.

| Platform     | Operation | Sec. Bits | Bits/Cycle | Bits/ $\mu$ J |
|--------------|-----------|-----------|------------|---------------|
| ESP32-C61    | KeyGen    | 128       | 1.36e-5    | 0.5464        |
| ESP32-C61    | Encap     | 128       | 1.22e-5    | 0.4926        |
| ESP32-C61    | Decap     | 128       | 1.03e-5    | 0.8333        |
| STM32 F446RE | KeyGen    | 128       | 1.26e-5    | 0.1330        |

| Platform       | Operation | Sec. Bits | Bits/Cycle | Bits/ $\mu$ J |
|----------------|-----------|-----------|------------|---------------|
| STM32 F446RE   | Encap     | 128       | 1.14e-5    | 0.1180        |
| STM32 F446RE   | Decap     | 128       | 9.59e-6    | 0.1020        |
| RP2040 Pico    | KeyGen    | 128       | 1.00e-5    | 0.0376        |
| RP2040 Pico    | Encap     | 128       | 8.59e-6    | 0.0331        |
| RP2040 Pico    | Decap     | 128       | 7.09e-6    | 0.0278        |
| nRF52840 DK    | KeyGen    | 128       | 1.16e-5    | 0.4290        |
| nRF52840 DK    | Encap     | 128       | 1.01e-5    | 0.3982        |
| nRF52840 DK    | Decap     | 128       | 8.27e-6    | 0.5162        |
| Raspberry Pi 5 | KeyGen    | 128       | 5.02e-3    | —             |
| Raspberry Pi 5 | Encap     | 128       | 4.04e-3    | —             |
| Raspberry Pi 5 | Decap     | 128       | 3.05e-3    | —             |

# Appendix C

## Quantum Search Adjusted Symmetric Normalization

Table C.1: Security-normalized symmetric AEAD efficiency at 1024 bytes under a quantum-search-adjusted normalization.

| Platform     | Algorithm     | Sec. Bits | Bits/Cycle | Bits/ $\mu$ J |
|--------------|---------------|-----------|------------|---------------|
| ESP32-C61    | AES-128-GCM   | 64        | 3.84e-7    | 0.0767        |
| ESP32-C61    | Ascon-AEAD128 | 64        | 1.19e-6    | 12.8600       |
| ESP32-C61    | Ascon-80pq    | 80        | 1.14e-6    | 12.7999       |
| ESP32-C61    | GIFT-COFB     | 64        | 1.08e-6    | 11.1322       |
| STM32 F446RE | AES-128-GCM   | 64        | 4.74e-7    | 0.0017        |
| STM32 F446RE | Ascon-AEAD128 | 64        | 1.11e-6    | 0.3250        |
| STM32 F446RE | Ascon-80pq    | 80        | 1.07e-6    | 0.3701        |
| STM32 F446RE | GIFT-COFB     | 64        | 1.14e-6    | 0.3221        |
| RP2040 Pico  | AES-128-GCM   | 64        | 3.31e-7    | 0.0008        |
| RP2040 Pico  | Ascon-AEAD128 | 64        | 6.98e-7    | 0.1367        |
| RP2040 Pico  | Ascon-80pq    | 80        | 6.64e-7    | 0.1289        |
| RP2040 Pico  | GIFT-COFB     | 64        | 6.88e-7    | 0.1544        |
| nRF52840 DK  | AES-128-GCM   | 64        | 3.27e-7    | 0.0067        |

| Platform       | Algorithm     | Sec. Bits | Bits/Cycle | Bits/ $\mu$ J |
|----------------|---------------|-----------|------------|---------------|
| nRF52840 DK    | Ascon-AEAD128 | 64        | 9.72e-7    | 2.1587        |
| nRF52840 DK    | Ascon-80pq    | 80        | 9.56e-7    | 4.5246        |
| nRF52840 DK    | GIFT-COFB     | 64        | 1.04e-6    | 6.6809        |
| Raspberry Pi 5 | AES-128-GCM   | 64        | 6.97e-5    | —             |
| Raspberry Pi 5 | Ascon-AEAD128 | 64        | 4.09e-4    | —             |
| Raspberry Pi 5 | Ascon-80pq    | 80        | 4.20e-4    | —             |
| Raspberry Pi 5 | GIFT-COFB     | 64        | 1.24e-4    | —             |